Definition der Umsteigebeziehungen von Flixbus und BlaBlaCar zum öffentlichen Verkehr

(Hauptstudie)

Autorin: Blerta Imeri
Kleiner Muristalden 40
CH-3006 Bern
blerta.imeri@stud.hslu.ch

Betreuer: Prof. Dr. Timo Ohnmacht
Hochschule Luzern – Wirtschaft
Rösslimatte 48
6002 Luzern
timo.ohnmacht@hslu.ch

Co-Betreuer: Matthias Günter
SBB
Wylerstrasse 123
CH-3014 Bern
matthias.guenter@sbb.ch

Hochschule Luzern
Master of Science in Applied Information and Data Science (MScIDS)
Herbstsemester 2025

Bern, 19. Dezember 2025

Zusammenfassung

Durch das stetig wachsende Transportangebot in der Schweiz, ist es wichtig, dass der öffentliche Verkehr für die Nutzerinnen und Nutzer möglichst einfach zugänglich bleibt. Besonders für Verkehrsunternehmen spielt es eine grosse Rolle, neue Haltestellen flexibel in bestehende Angebote integrieren zu können. Für diesen Prozess stehen verschiedene Ansätze und Werkzeuge zur Verfügung.

Aktuell sind im Schweizer öV-System die Fernverkehrshaltestellen von FlixBus und BlaBlaCar nicht enthalten. Um diese dennoch berücksichtigen zu können, müssen die Haltestellen über sogenannte Umsteigebeziehungen, welche die Verbindung in Gehminuten und Gehweg zwischen Haltestellen abbildet, in das System von *opentransportdata.swiss* integriert werden.

Für die Berechnung der Umsteigebeziehungen werden im ersten Schritt die Haltestellendaten der Fernverkehrsanbieter anhand eines ETL-Prozesses vorbereitet. Mittels dieser werden anhand von drei Methoden in QGIS und einer in Python die Gehwege und Gehzeiten berechnet, um die nächstgelegene Haltestelle zu finden. Die Gehzeiten werden danach mit aktuellen Geodaten aus Google Maps verglichen.

Der zum Abschluss durchgeführte Vergleich zeigt, dass die Unterschiede der berechneten Gehminuten zwischen den untersuchten Methoden gering sind und sich im Durchschnitt zwischen null und einer Minute bewegen. Bei der Analyse der insgesamt 30 nächstgelegenen Haltestellen, werden fünfmal abweichende Zielhaltestellen und 25 mal exakt dieselbe ermittelt. Diese Unterschiede lassen sich dadurch erklären, dass trotz gleichem Wegenetz nicht überall die gleichen Strassen oder Wege berücksichtigt werden und die berechneten Routen teilweise an leicht unterschiedlichen Stellen beginnen oder enden.

Insgesamt zeigt sich, dass sich insbesondere automatisierte Python-Lösungen gut für eine zukünftige, wiederholbare Anwendung eignen, während manuelle GIS-Methoden vor allem für visuelle Kontrolle hilfreich sind.

Inhaltsverzeichnis

Abbildungsverzeichnis IV
Tabellenverzeichnis V
Abkürzungsverzeichnis VI
1 Ziel der Arbeit 1
1.1 Relevanz / Problem 1
1.2 Forschungsfragen 1
1.3 Lösungsansatz 1
2 Stakeholder 2
2.1 SKI+ 2
2.2 FlixBus und BlaBlaCar 2
3 Angewandte Programme und Programmiersprache 3
3.1 QGIS 3
3.2 GitHub 3
3.3 VS-Code / Python 3
4 Umsteigebeziehungen 4
4.1 Definition 4
4.2 Aktuelle Berechnungsgrundlage 4
4.3 Beispiele für benötigte Umsteigebeziehungen 4
4.4 Aufbau der HRDF Formate 5
5 ETL 9
5.1 Extract 9
5.2 Transform 9
5.3 Load 10
5.4 Readme file von GitHub 10
6 Methoden zur Berechnung der Umsteigebeziehungen 17
6.1 Netzwerk-Analyse 17
6.2 QNEAT3 Plugin 18
6.2.1 Anwendungsbeispiel 18
6.2.2 Analyse 19
6.3 ORS Tools Plugin 21
6.3.1 Anwendungsbeispiel 22
6.3.2 Analyse 22
6.4 OSMnx anhand Python 25
6.4.1 Anwendungsbeispiel 25

6.4.2 Analyse....26
6.5 Methodenvergleich....28
7 Beantwortung der Forschungsfragen....33
8 Ergebnis und Ausblick....34
Literaturverzeichnis....35
Anhang....37

Abbildungsverzeichnis

Abbildung 1: Situationsplan Bern Neufeld 4
Abbildung 2: Situationsplan Zürich Hauptbahnhof 5
Abbildung 3: Projektstruktur des GiHub Repositorys 12
Abbildung 4: Vergleich Gehminuten: QNEAT3 vs. Google Maps 19
Abbildung 5: Delta (QNEAT3 - Google Maps) 20
Abbildung 6: Vergleich Gehminuten: ORS Tools vs. Google Maps 23
Abbildung 7: Delta (ORS Tools - Google Maps) 24
Abbildung 8: Vergleich Gehminuten: OSMnx vs. Google Maps 26
Abbildung 9: Delta (OSMnx - Google Maps) 27
Abbildung 10: OSMnx Strecke Erstfeld Gotthard Service Station South 29
Abbildung 11: ORS Strecke Erstfeld Gotthard Service Station South 29
Abbildung 12: OSMnx Strecke Zurich Bus Station 30
Abbildung 13: ORS Tools Strecke Zurich Bus Station 30
Abbildung 14: OSMnx Strecke Chur station train station (Bahnhofplatz) 30
Abbildung 15: ORS Tools Strecke Chur station train station (Bahnhofplatz) 30
Abbildung 16: OSMnx Strecke St. Gallen train station 31
Abbildung 17: ORS Tools Strecke St. Gallen train station 31
Abbildung 18: OSMnx Strecke Luzern (Landenberg) 31
Abbildung 19: ORS Tools Strecke Luzern (Landenberg) 31

Tabellenverzeichnis

Tabelle 1: Präzisierung für Haltestellengruppen.....6
Tabelle 2: Verteilung der absoluten Abweichungen von QNEAT3 21
Tabelle 3: Zusammenfassung der Abweichungen (in Minuten) von QNEAT3 21
Tabelle 4: Verteilung der absoluten Abweichungen von ORS Tools 24
Tabelle 5: Zusammenfassung der Abweichungen (in Minuten) von ORS Tools 25
Tabelle 6: Verteilung der absoluten Abweichungen von OSMnx 28
Tabelle 7: Zusammenfassung der Abweichungen (in Minuten) von OSMnx 28
Tabelle 8: Zeitliche Unterschiede zwischen OSMnx, ORS Tools und Google Maps 29

Abkürzungsverzeichnis

API	Application Programming Interface
ETL	Extract, Transform, Load
GTFS	General Transit Feed Specification
HRDF	HAFAS Rohdaten Format
ORS	OpenRouteService
OSM	OpenStreetService
öV	öffentliches Verkehrsmittel
QGIS	Quantum GeoInformationSystem
SFTP	Secure File Transfer Protocol
VS Code	Visual Studio Code

1 Ziel der Arbeit

Das Ziel der Arbeit ist es, die Umsteigebeziehungen zwischen dem öffentlichen Verkehr (öV) der Schweiz und den Fernverkehrshaltestellen von Flixbus und BlaBlaCar herzustellen.

1.1 Relevanz / Problem

Aktuell fehlen jegliche Umsteigebeziehungen zwischen den Haltestellen der beiden Fernverkehrsanbieter und den öV-Haltestellen der Schweiz. Die Umsteigebeziehungen werden jedoch benötigt, um die Fernverkehrshaltestellen ins *opentransportdata.swiss* zu integrieren. Da die zu integrierenden Datensätze mit den Fernverkehrshaltestellen aktuell nur als General Transit Feed Specification (GTFS) Format vorliegen, die öffentlich zugänglichen Haltestellen im System von *opentransportdata.swiss* jedoch als HAFAS Rohdaten Format (HRDF), ist eine Integration ohne Umwandlung der Datensätze aktuell nicht möglich.

1.2 Forschungsfragen

Im Rahmen dieser Arbeit stehen die technische Verarbeitung sowie die Integration von Fernverkehrsdaten in das bestehende öffentliche Verkehrssystem der Schweiz im Mittelpunkt. Daraus ergeben sich die folgende zentrale Forschungsfragen:

- Wie können die GTFS-Daten der Fernverkehrshaltestellen technisch verarbeitet und in das *opentransportdata.swiss*-System integriert werden?
- Wie lassen sich Fusswegdistanzen und Umsteigezeiten zwischen den Fernverkehrshaltestellen und den bestehenden öV-Haltestellen berechnen (Umsteigebeziehungen)?

Zur Beantwortung dieser Hauptfragen werden zusätzliche Unterfragen betrachtet, die eine genauere Analyse ermöglichen:

- Welche Kriterien definieren eine sinnvolle Umsteigebeziehung (z. B. maximale Distanz, Gehzeit, Zugänglichkeit, Slack/Zusatzzeit für Gepäckausladen)?
- Wie lässt sich die nächstgelegene öV-Haltestelle zu einer Fernverkehrshaltestelle geografisch ermitteln?

1.3 Lösungsansatz

Im ersten Schritt werden die Fernverkehrshaltestellen mit Python in Visual Studio Code (VS Code) so aufbereitet, dass diese der Struktur der Dateien `BAHNHOF` und `BFKOORD_WGS` aus *opentransportdata.swiss* entsprechen. Dazu gehört das Entfernen nicht benötigter Spalten, das Ergänzen einer ID für alle Haltestellen, die mit dem Format der öV-Haltestelle übereinstimmt, und das Entfernen aller sich nicht in der Schweiz befindlichen Haltestellen. Zusätzlich müssen Duplikate identifiziert und bereinigt werden. Im Rahmen dieser Arbeit liegt ein Duplikat vor, wenn sich die nächstgelegene Haltestelle innerhalb eines 100 m Radius befindet.

Danach werden anhand der bereinigten Dateien die Umsteigebeziehungen berechnet und im Format `METABHF` und `UMSTEIGB` ausgegeben, sodass diese ins *opentransportdata.swiss* integriert werden können. Für die Berechnung der Umsteigebeziehungen existieren mehrere mögliche Vorgehensweisen. In dieser Arbeit werden drei Methoden in QGIS und eine in Python untersucht und deren Ergebnisse verglichen. Ziel ist es eine Methode zu finden, die für den späteren Einsatz im Betrieb verständlich, nachvollziehbar und ohne grossen Aufwand wiederholbar ist.

2 Stakeholder

In den folgenden Unterkapiteln werden die Stakeholder beschrieben.

2.1 SKI+

Der Auftraggeber dieser Arbeit, SKI+, ist Teil der SKI (Systemaufgabe Kundeninformation) und unterstützt das BAV (Bundesamt für Verkehr) bei der Vorbereitung der NADIM (Nationale Datenvernetzungsinfrastruktur Mobilität) (Daten für ein effizientes Mobilitätssystem - BAV, 2025), einer Initiative für effiziente, klimafreundliche, inklusive und sichere Mobilität mit EU-Anbindung. Durch SKI+ und Partner/Lieferanten wurde die Plattform Open-Data-Plattform Mobilität Schweiz (ODMCH) ins Leben gerufen (Über SKI+ | öv-info.ch, o. J.).

Die in der Arbeit ermittelten Daten, werden von SKI+ anschliessend im *opentransportdata.swiss* öffentlich zugänglich gemacht. Dabei ist wichtig, dass nur dann korrekte und vollständige Informationen bereitgestellt werden können, wenn alle relevanten Haltestellen miteinander verknüpft sind. Ohne diese Umsteigebeziehungen können die Fernverkehrsangebote nicht korrekt im System abgebildet werden, was sowohl die Datenqualität als auch die Nutzbarkeit für externe Anwendungen einschränkt. Die in dieser Arbeit erstellten Verbindungen schaffen daher die Grundlage dafür, dass SKI+ diese Fernverkehrsdaten fachgerecht integrieren und zur Verfügung stellen kann.

2.2 FlixBus und BlaBlaCar

FlixBus ist neben FlixTrain, Greyhound und Kâmil Koç, eine Marke des globalen Travel-Tech-Unternehmens Flix. Dieser bietet Reisemöglichkeiten in über 40 Ländern auf 4 Kontinenten an (Über Flix: Fernbus- und Zugbetreiber, o. J.).

BlaBlaCar wurde 2006 als Mitfahrplattform gegründet. Siebietet Fahrenden und Mitfahrenden, die das selbe Reiseziel haben, eine Möglichkeit sich miteinander zu vernetzen. Nach der Übernahme von Ouibus im Jahr 2019 erweiterte das Unternehmen ihr Angebot mit BlaBlaCar Bus, um zusätzliche günstige und komfortable Busreisen auf ihrer Plattform anbieten zu können (Entdecken Sie BlaBlaCar Bus, o. J.).

3 Angewandte Programme und Programmiersprache

In diesem Kapitel werden die Programmiersprache und die verwendeten Programme beschrieben, welche für die Beantwortung der Forschungsfragen verwendet wurden.

3.1 QGIS

QGIS (ehemals Quantum GIS) ist ein Open-Source-Geoinformationssystem, welches die Erstellung, Bearbeitung, Visualisierung, Analyse und Veröffentlichung geografischer Informationen ermöglicht. Ein zentraler Vorteil von QGIS besteht darin, dass die Software kostenlos zur Verfügung steht und von einer grossen Entwickler- und Nutzercommunity kontinuierlich weiterentwickelt wird. Darüber hinaus bietet QGIS eine breite Auswahl an Plugins, mit denen sich die Funktionen flexibel erweitern und an spezifische Anforderungen anpassen lassen. Zu den wichtigsten Einsatzbereichen gehören die Datenbearbeitung und Datenverwaltung, die Erstellung und Visualisierung von Karten, räumliche Analysen sowie die Integration von Datenbanken und anderen Formaten (maptransfer, o. J.).

Für diese Arbeit wurde die Version 3.40 von QGIS verwendet. Sie gehört nicht zu den experimentellen Versionen, sondern stellt eine stabile und offiziell empfohlene Ausgabe dar. Sie eignet sich besonders für Arbeiten, bei welchen zuverlässige Funktionen und eine hohe Kompatibilität mit Plugins erforderlich sind. Die verwendete Version kann über die offizielle QGIS-Webseite heruntergeladen werden (QGIS, o. J.).

3.2 GitHub

GitHub ist eine cloudbasierte Plattform zum Speichern, Teilen und Zusammenarbeiten von Codes. Mit dem Speichern von Codes in Repositories auf Github wird ermöglicht, dass die Arbeit einerseits präsentiert oder geteilt und andererseits der Code im Laufe der Zeit nachverfolgt und verwaltet werden kann. Des Weiteren können andere den Code überprüfen, welches eine Zusammenarbeit im Projekt ermöglicht. Git ist ein Versionskontrollsystem, mit dem Änderungen an Dateien intelligent nachverfolgt werden können. Während Git ermittelt, wie der Fluss von Änderungen intelligent zusammengeführt werden kann, hilft GitHub beim Verwalten des Flusses (GitHub-Dokumentation, o. J.).

Die GitHub Plattform wurde vom Auftraggeber als zentrale Umgebung für den Austausch und die Bereitstellung des Codes und der zugehörigen Dateien festgelegt.

3.3 VS-Code / Python

VS Code ist ein Quellcode-Editor von Microsoft, welcher kostenlos zur Verfügung steht. Mit Hilfe des Programms lassen sich Programmiersprachen, wie unter anderem Python, JavaScript oder C++, bearbeiten. Dank einer grossen Auswahl an Erweiterungen kann der Editor für viele weitere Sprachen, Entwicklungsumgebungen und Cloud-Dienste erweitert werden (Heller, 2024).

Eine zentrale Anforderung des Auftraggeber ist, dass der gesamte Code in Python umgesetzt wird. Python ist eine anfängerfreundliche und objektorientierte Programmiersprache, welche mit einer leicht lesbaren Syntax einfach zu interpretieren ist. Diese Sprache wurde 1989 von Guido van Rossum geschrieben und fünfeinhalb Jahre später veröffentlicht. Mit Python können komplexe Aufgaben, welche vereinfacht werden müssen, in kurzen Skripts dargestellt werden (Oracle, o. J.).

4 Umsteigebeziehungen

Nachfolgend wird die Definition der Umsteigebeziehung erläutert.

4.1 Definition

Die Umsteigebeziehung setzt sich zusammen aus der Umsteigezeit und dem Umsteigeweg. Dabei ist die Umsteigezeit die "[…] minimale Zeit zwischen Ankunft eines Anschlussgebers und Abfahrt eines Anschlussnehmers, damit ein Anschluss innerhalb des gleichen Betriebspunkts gewährleistet ist und dem Kunden publiziert wird. Diese Zeit kann pro Betriebspunkt definiert werden. Der Standardwert liegt bei 2 Minuten." (Oev-info, 2020). Umsteigeweg ist die Distanz, die ein Reisender zu Fuss zwischen zwei Anschlussstellen im öffentlichen Verkehr zurücklegen musste, und gilt als Mass für die Qualität einer Umsteigebeziehung (VÖV UTP, 2024).

Da die Fernverkehrsanbieter jedoch nicht an genau denselben Orten halten, wie beispielsweise Züge an einem Bahnhof, reicht die allgemeine Standard-Umsteigezeit nicht aus. Für diese Fälle muss die tatsächliche Gehzeit zwischen den Haltestellen zusätzlich berücksichtigt werden.

4.2 Aktuelle Berechnungsgrundlage

Laut dem Auftraggeber werden Umsteigebeziehungen zwischen bestehenden Haltestellen derzeit manuell erstellt. Das heisst konkret: Wenn eine neue Haltestelle hinzugefügt werden soll, wird diese einzeln geprüft. Im Anschluss wird manuell festgelegt, welche Haltestelle die nächstgelegene Umsteigestation ist. Die gesamte Berechnung basiert demnach auf einer nicht automatisierten Einschätzung.

4.3 Beispiele für benötigte Umsteigebeziehungen

Um die Wichtigkeit des Projektes zu verstehen, wird anhand von zwei Beispielen aufgezeigt, wo sich aktuell die Fernverkehrshaltestellen befinden und wie diese mit den öV-Haltestellen verknüpft werden müssen.

Der Situationsplan von Bern Neufeld sieht wie folgt aus:



Abbildung 1: Situationsplan Bern Neufeld

Der grüne Punkt zeigt die aktuelle Haltestelle des öV, der blaue Punkt die Haltestelle des Fernverkehrs. Wie aus der Abbildung hervorgeht, liegen diese nicht am gleichen Standort.

Die gleiche Situation zeigt sich am Hauptbahnhof in Zürich.

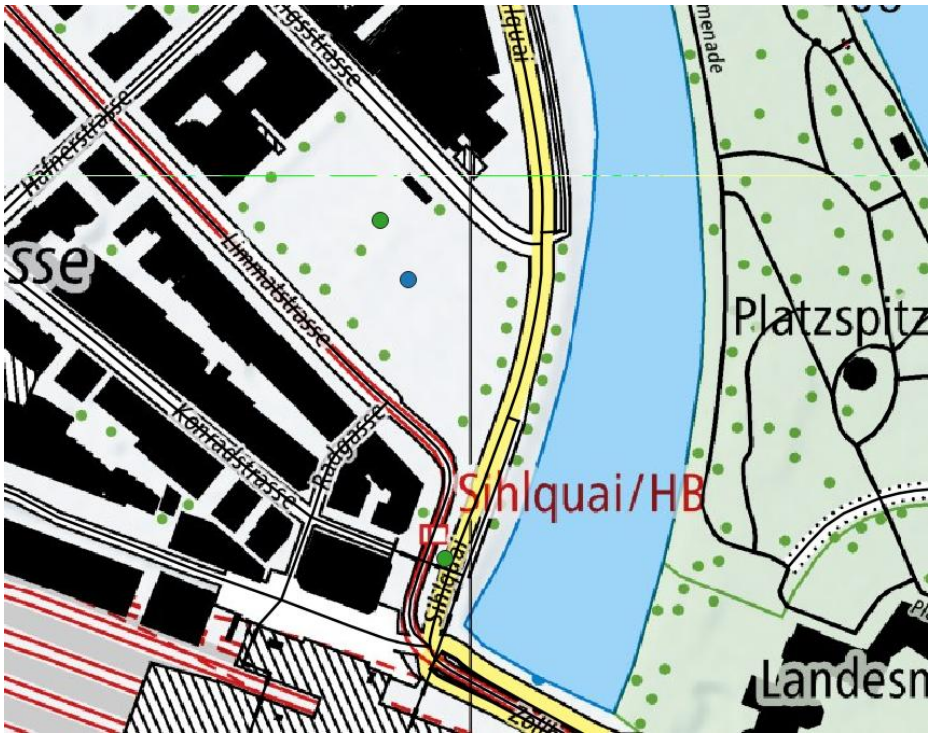


Abbildung 2: Situationsplan Zürich Hauptbahnhof

Auch hier wird die öV-Haltestelle in grün angezeigt und die des Fernverkehrsunternehmens in blau.

Anhand dieser Beispiele wird die Relevanz der Umsteigebeziehungen dargestellt, da die Fernverkehrshaltestelle und öV-Haltestelle nicht in unmittelbarer nähe zueinander liegen.

4.4 Aufbau der HRDF Formate

Wie in Kapitel 1.3 beschrieben, werden vom Auftraggeber zwei Dateien benötigt, `METABHF` und `UMSTEIGB`, welche beide im HRDF Format aufgebaut sind. Dieses ist ein proprietäres Datei-Format, welches einzelne Dateien enthält, die einzelnen Tabellen bzw. Objekten entsprechen (opentransportdata.swiss, o. J.). Im öffentlich verfügbaren Dokument vom SKI wird wie folgt beschrieben, wie die einzelnen Tabellen aufgebaut sind (HRDF-Realisierungsvorgaben-öV-Schweiz, 2024).

Die `METABHF`-Datei ist in zwei Teile aufgebaut.

> **6.10 Verbindungen zwischen Haltestellen `METABHF`**
>
> Format entspricht der Norm [1].
>
> *Bemerkungen:*
> - Quellsystem: Das Quellsystem für Fusswege ist INFO+. Die Meta-Bahnhof Informationen werden in INFO+ nicht importiert.
> - Granularität: Fusswege werden auf Stufe Haltestelle definiert.

6.10.1 Übergangsbeziehungen

Format entsprich der Norm [1]

***A-Zeilen (optional)**

Format entsprich der Norm [1]

Beispiel:

```
0012105 0100020 005
*A B1
```

Der obere Teil besteht aus Übergangsbeziehungen wie in Punkt 6.10.1 beschrieben. Wobei der untere Teil die Haltestellengruppen darstellt.

6.10.2 Haltestellengruppen

Format entsprich der Norm [1]

Tabelle 8: Präzisierung für Haltestellengruppen

Tabelle 1: Präzisierung für Haltestellengruppen

Spalte	Typ	Bedeutung	Hinweis
1.7	INT32	Die Nummer des Sammelbegriffs.	
8-8	CHAR	Festes Zeichen ‚:'.	
10-10	CHAR	Der Typ der 1. Äquivalenz: S, B, F, V, H oder Leerzeichen.	Zurzeit wird Leerzeichen geliefert
11-17	INT31	Die Nummer der 1. Äquivalenz	
19-19	CHAR	Der Typ der 2. Äquivalenz: S, B, F, V, H oder Leerzeichen.	Zurzeit wird Leerzeichen geliefert
20-26	INT32	Die Nummer der 2. Äquivalenz.	
28ff		Typ und Nummer der nächsten Äquivalenz … usw.	

Beispiel:

```
8512105: 8512105 8500020
```

Die zweite gewünschte Datei ist definiert als Haltestellenbezogene Umsteigezeiten `UMSTEIGB` und ist wie folgt aufgebaut:

8 Umsteigeinformation

8.1 Haltestellenbezogene Umsteigezeiten **`UMSTEIGB`**

Format entspricht der Norm [1].

Beispiel:

```
9999999 05 05 STANDARD
5800010 04 04 Basel SBB
```

Bemerkung:

Import / Export: Wird nur exportiert und nicht importiert.

Um diese zwei Formate erstellen zu können, werden zwei weitere Dateien benötigt. Einerseits das Haltestellenverzeichnis `BAHNHOF`:

6 Haltestellendaten

6.1 Das Haltestellenverzeichnis `BAHNHOF`

Format entspricht der Norm [1].

Tabelle 4: Präzisierung `BAHNHOF`

Spalte	Typ	Bedeutung	Hinweis
1-7	INT32	Die Nummer der Haltestelle	DiDok-Nr
9-11	CHAR	Leer	
13ff	CHAR	Haltestellenname	Export: Haltestellennamen werden mit einem Typ geliefert (Typ in spitzen Klammern folgen hinter dem Item): <1> Name (max. 30 Zeichen) <2> Langname (max. 50 Zeichen) <3> Abkürzung <4> Synonym / Aliase Sprachabhängige Namen werden nicht unterstützt.

Beispiel:

```
8507000    Bern$<1>$BN$<3>
8507002    Ostermundigen$<1>
8501026    Genève-Aéroport$<1>$GEAP$<3>$Geneva Air-
port$<4>$Genf Flughafen$<4>$Ginevra Aeroporto$<4>
```

Bemerkungen:

- DiDok 3 ist das Mastersystem für Haltestellennamen und -nummern.
- DiDok-Nummern (gemeint sind die 7-stelligen Nummern mit dem Ländercode 85 am Anfang): Haltestellen, an denen Fahrgäste aus- oder einsteigen können, müssen eine gültige DiDok-Nr haben. Touristischen Via-Informationen (fiktive Vias), Meta-BP und Grenzpunkte sind von dieser Regel ausgenommen, die Anwendung dieser Regel wird aber auch für diese Objekte empfohlen.
- Haltestellen-Aliase: Kommen nicht aus DiDok, sondern werden in INFO+ gepflegt.

Wie auch die Haltestellendaten im Format `BFKOORD_WGS`:

- `BFKOORD_WGS`: Koordinaten im WGS84-Format. Weltweit gültig. Format siehe Kapitel 6.2.3

6.2.3 BFKOORD_WGS

Format entspricht der Norm [1].

Tabelle 6: Folgende Präzisierungen gelten bei BFKOORD_WGS

Spalte	Typ	Bedeutung	Hinweis
1-7	INT32	Die Nummer der Haltestelle	
9-19	FLOAT	X-Koordinate, geografische Länge	Format WGS84, [-]NNN.nnnnnn, rechtsbündig
21-31	FLOAT	Y-Koordinate, geografische Breite	Format WGS84, [-]NN.nnn-nnn, rechtsbündig
33-38	INT16	Z-Koordinate	Höhe über Meer in Metern, linksbündig (kann leer sein)
40ff	CHAR	Kommentar (mit führendem %)	Optional. Nur zur besseren Lesbarkeit

Beispiel:

```
8570203 6.637803 46.640402 622 % Echallens, place Emile Gar-
daz
8570204 6.642109 46.640720 617 % Echallens, La Robellaz
8570238 6.632576 46.639735 617 % Echallens, gare
```

Bemerkung:

- WGS84 ist ein Koordinatensystem mit Längen- und Breitengraden.

Wie bereits erwähnt, werden beide Formate benötigt, um zum gewünschten Ergebnis zu gelangen. Durch das Erstellen des BAHNHOFS-Formats erhalten alle Fernverkehrshaltestellen eine eindeutige ID. Diese dient als Grundlage, um anschliessend das BFKOORD_WGS-Format zu erstellen. Anhand der darin enthaltenen Koordinaten können die Standorte der Haltestellen bestimmt und die entsprechenden Umsteigebeziehungen berechnet werden.

Dabei ist zu beachten, dass die Fernverkehrsanbieter keine Z-Koordinaten zur Verfügung gestellt haben. Aus diesem Grund wird die Höheninformation in dieser Arbeit nicht berücksichtigt.

5 ETL

In diesem Abschnitt werden die ETL-Schritte (Extract, Transform, Load) beschrieben, welche durchgeführt wurden, um die Haltestellen der Fernverkehrsanbieter so aufzubereiten, dass sie in das bestehende öV-System der Schweiz integriert werden können.

5.1 Extract

Zu Beginn lädt die Pipeline alle benötigten Daten automatisch herunter. Dazu gehören die HRDF-Referenzdaten von *opentransportdata.swiss* (`BAHNHOF`, `BFKOORD_WGS`, `METABHF`, `UMSTEIGB`), die GTFS-Haltestellendateien der Fernverkehrsanbieter, sowie die Schweizer Landesgrenzen aus *swissBOUNDARIES3D* (opendata.swiss, 2024). Die HRDF-Daten dienen als Grundlage, um bestehende öV-Haltestellen zu erkennen und Duplikate zu vermeiden. Die GTFS-Daten liefern die Rohinformationen zu den Fernverkehrshaltestellen.

Für *opentransportdata.swiss* und Flixbus sind Permalinks verfügbar, wodurch der Download automatisch erfolgen kann und nicht bei jeder Änderung von Hand angepasst werden muss. Für BlaBlaCar existiert kein Permalink, weshalb der aktuell verfügbare Downloadlink verwendet wird. Die Pipeline legt alle heruntergeladenen Dateien im Ordner *data/raw* ab. Zusätzlich wird über einen *MD5-Hash* geprüft, ob sich eine Datei seit dem letzten Lauf verändert hat. Falls nicht, wird die Verarbeitung übersprungen, um die Anzahl der Arbeitsschritte gering zu halten.

Nach dem Download werden die Haltestellen mithilfe der Swissboundaries-Daten auf das Schweizer Staatsgebiet eingeschränkt. Haltestellen ausserhalb des Landes werden dabei automatisch ausgeschlossen. Dadurch bleiben nur diejenigen Haltestellen übrig, die für diese Arbeit relevant sind.

5.2 Transform

Im Transform-Schritt werden die Daten bereinigt und so aufbereitet, dass spätere Berechnungen und Integrationen problemlos möglich sind. Zuerst erfolgt die Entfernung nicht benötigter Einträge, zum Beispiel aller FlixTrain-Haltestellen, da diese bereits im bestehenden öV-Netz enthalten sind. Anschliessend werden räumliche Duplikate erkannt und entfernt. Die vordefinierte 100 Meter Distanz wird über die Haversine-Formel berechnet, die die Grosskreisentfernung zwischen zwei Koordinaten bestimmt. Zusätzlich werden die bereinigten Fernverkehrshaltestellen mit den bestehenden Koordinaten in `BFKOORD_WGS` verglichen, um Haltestellen zu entfernen, die bereits im öV-System vorhanden sind.

Für alle verbleibenden Haltestellen wird eine neue eindeutige ID vergeben. Dazu wurde ein eigener Nummernbereich von 1700000–1799999 definiert. Die ersten beiden Ziffern dieser IDs sind bewusst künstlich gewählt und dienen der eindeutigen Abgrenzung innerhalb des Systems. Dadurch werden Kollisionen mit bestehenden IDs vermieden und die Fernverkehrshaltestellen klar als separate Kategorie gekennzeichnet. Die eindeutigen IDs werden erst nach der Bereinigung vergeben, damit keine Lücken entstehen.

Zum Abschluss des Transform-Schritts werden die Daten im `BFKOORD_WGS`-Format geschrieben (ID, Koordinaten, Name) und zusätzlich als kommagetrennte *csv*-Datei exportiert. Diese dienen dann als Grundlage für die QGIS-Analysen zur Berechnung der Umsteigebeziehungen.

5.3 Load

Nach dem Transform Schritt erfolgt der Load-Schritt. Hier werden alle aufbereiteten Daten in die endgültigen Zielstrukturen überführt. Aus den bereinigten Fernverkehrshaltestellen werden eine `BFKOORD_WGS`-Datei sowie die dazugehörige `BAHNHOF`-Datei erzeugt. Beide entsprechen dem HRDF-Format und können als „Delta" zum bestehenden öV-Netz betrachtet werden. Die Dateien werden im Ordner *data/processed/delta* abgelegt.

Zusätzlich werden `METABHF`- und `UMSTEIGB`-Dateien erstellt, die die neuen Umsteigebeziehungen und Umsteigezeiten enthalten. Je nach Methode entstehen diese entweder automatisch über das OSMnx Packet oder über eine manuelle QGIS-Methode, welche anschliessend wieder in die Code Pipeline eingelesen werden. Schlussendlich werden alle Delta-Dateien zu einem ZIP-Archiv zusammengefasst, sodass diese sie einfacher an SKI+ weitergeben zu können.

Der gesamte ETL-Prozess ist als Open-Source-Projekt auf GitHub veröffentlicht, sodass der Prozess reproduzierbar bleibt und auch in Zukunft erweitert oder angepasst werden kann. Das GitHub Repository ist unter folgendem Link abrufbar:
https://github.com/openTdataCH/generate_transfer_stops_coach_public_transport

5.4 README-Datei aus GitHub

Zum besseren Verständnis des Python Projektes, wird nachfolgend die README-Datei abgebildet.

Generate Transfer Stops Coach Public Transport

ETL-Pipeline zur automatischen Verarbeitung von Fernverkehrshaltestellen (Flixbus, BlaBlaCar) und deren Integration in das öffentliche Verkehrsnetz der Schweiz.

Übersicht

Das Projekt lädt automatisch GTFS-Daten von Fernverkehrsanbietern und öV-Referenzdaten herunter, filtert Haltestellen in der Schweiz, bereinigt Duplikate und bereitet sie für die Integration in bestehende öV-Systeme auf.

ETL-Pipeline:

- **Extract**: Automatischer Download von öV-Referenzdaten und GTFS-Daten, Filterung auf Schweizer Haltestellen
- **Transform**: Koordinatensammlung, Datenbereinigung (FlixTrain-Entfernung, räumliche Duplikate), ID-Vergabe
- **Load**: Erzeugung von Output-Dateien im BFKOORD_WGS und BAHNHOF-Format

Wichtige Ressourcen

Datenquellen

- **ÖV-Referenzdaten Schweiz:** opentransportdata.swiss BAHNHOF, BFKOORD_WGS, METABHF, UMSTEIGB
- **Flixbus GTFS:** Haltestellen-Daten im GTFS-Format
- **BlaBlaCar GTFS:** Haltestellen-Daten im GTFS-Format
- **Swiss Boundaries:** swisstopo swissBOUNDARIES3D Schweizer Landesgrenzen

Formatbeschreibungen

- **BFKOORD_WGS Format:** `ID LON LAT % NAME` Standard-Koordinatenformat für öV-Haltestellen
- **BAHNHOF Format:** `ID (8 Zeichen linksbündig) + 5 Leerzeichen + NAME$<1>` Legacy-Format für öV-Systeme
- **GTFS (General Transit Feed Specification):** gtfs.org Standard für öffentliche Verkehrsdaten
- **METABHF Format:** Umsteigebeziehungen zwischen Haltestellen im Format `ID1 ID2 Distanz` bzw. `ID2 : ID1`
- **UMSTEIGB Format:** `ID NUM NUM NAME` Umsteigezeiten zwischen Haltestellen, NUM = letzte 2 Ziffern der Gehzeit in Minuten aus METABHF

Tools & Algorithmen

- **QGIS:** qgis.org Open Source GIS-Software für manuelle Datenverarbeitung
- **Haversine-Formel:** Berechnung der Großkreis-Distanz zwischen GPS-Koordinaten auf einer Kugel
    - Verwendet für: Erkennung räumlicher Duplikate (< 100m Schwellwert)

ID-Vergabe

Warum 17xxxxx

- ÖV-Haltestellen-IDs in der Schweiz folgen dem UIC-System (8-stellig, beginnend mit 85)
- Fernverkehrshaltestellen sind NICHT Teil des regulären öV-Netzes
- ID-Bereich **1700000-1799999** wurde gewählt um:
    - Kollisionen mit bestehenden öV-IDs zu vermeiden
    - Fernverkehrshaltestellen klar vom öV-Netz zu unterscheiden
    - Raum für bis zu 100.000 neue Haltestellen zu schaffen

Projektstruktur

```
├ main.py                          # Haupteinstiegspunkt der Pipeline
├ requirements.txt                 # Python-Abhängigkeiten
├ README.md
├ .gitignore
├ data/
│  ├ external/                     # Externe Referenzdaten
│  │  └ swissBOUNDARIES3D_1_5_LV95_LN02.gpkg  # Schweizer Landesgrenzen (GeoPackage)
│  ├ processed/                    # Verarbeitete Output-Dateien
│  │  ├ delta/                     # Neue/geänderte Daten
│  │  │  ├ BFKOORD_WGS             # Neue Haltestellen mit IDs
│  │  │  ├ BAHNHOF                 # BAHNHOF-Format Output
│  │  │  ├ METABHF                 # METABHF mit Umsteigebeziehungen
│  │  │  └ UMSTEIGB                # Umsteigezeiten
│  │  ├ delta.zip                  # Gezippte Delta-Dateien für Weitergabe
│  │  ├ provider_stops             # Konsolidierte Haltestellen aller Provider
│  │  ├ BFKOORD_WGS_KOMMAGETRENNT.csv      # CSV der neuen Haltestellen
│  │  ├ QGIS_METABHF.csv                   # METABHF aus QGIS (manuell erstellt)
│  │  └ OEV_BFKOORD_WGS_KOMMAGETRENNT.csv  # ÖV-Referenzkoordinaten als CSV
│  └ raw/                          # Rohdaten (automatisch heruntergeladen)
│     ├ BlaBlaCar/
│     │  └ stops.txt               # GTFS stops.txt
│     ├ Flixbus/
│     │  └ stops.txt               # GTFS stops.txt
│     └ oevSammlung/              # ÖV-Referenzdaten Schweiz
│        ├ BAHNHOF
│        ├ BFKOORD_WGS
│        ├ METABHF
│        └ UMSTEIGB
└ src/
   └ transfer_stops/
      ├ __init__.py
      ├ config.py                  # Zentrale Konfiguration (Provider-URLs, IDs, Pfade)
      ├ clean_data.py              # Hilfsskript zum Löschen generierter Daten*
      ├ etl/                       # Extract-Transform-Load Module
      │  ├ __init__.py
      │  ├ extract.py              # Download (ÖV + GTFS) & Schweiz-Filterung
      │  ├ transform.py            # Datenbereinigung, ID-Vergabe & UMSTEIGB
      │  └ load.py                 # BAHNHOF-Format Generierung & ZIP-Erstellung
      └ create_metabhf/            # METABHF-Erstellung (OSM & QGIS)
         ├ __init__.py
         ├ create_osm_metabhf.py   # OSM-basierte METABHF-Generierung
         └ process_qgis_metabhf.py # Nachbearbeitung von QGIS METABHF CSV**
```

Abbildung 3: Projektstruktur des GiHub Repositorys

Hinweise:

- `*clean_data.py` ist ein Hilfsskript für Entwicklung/Testing, um generierte Daten zu löschen
- `**process_qgis_metabhf.py` ist ein Standalone-Script für QGIS-basierte METABHF-Verarbeitung
- Alle Pfade sind zentral in `config.py` konfiguriert

Installation

```
pip install pandas geopandas shapely requests
```

Verwendung

```
python main.py
```

Pipeline-Ablauf:

1. **Download öV-Referenzdaten:** Lädt BAHNHOF, BFKOORD_WGS, METABHF, UMSTEIGB von opentransportdata.swiss
    - Zweck: Referenzdaten um Duplikate mit bestehendem öV-Netz zu erkennen
2. **Download GTFS-Daten:** Lädt Flixbus und BlaBlaCar Haltestellen
    - Zweck: Erfassung aller Fernverkehrshaltestellen in Europa
3. **Änderungserkennung:** MD5-Hash-Vergleich, überspringt Verarbeitung wenn keine Änderungen
    - Zweck: Effizienz = Pipeline läuft nur bei tatsächlichen Datenänderungen
4. **Schweiz-Filterung:** Filtert nur Haltestellen innerhalb der Schweizer Landesgrenzen
    - Verwendet: swissBOUNDARIES3D von swisstopo
    - Grund: Fernverkehr hält auch in Deutschland/Italien/Österreich, wir wollen nur CH-Haltestellen
5. **Provider-Verarbeitung**: Sammelt neue Koordinaten pro Provider (ohne IDs)
    - Zweck: Zentrale Koordinatensammlung vor Bereinigung
6. **Datenbereinigung**:
    - **FlixTrain-Filter:** Entfernt alle Einträge mit "FlixTrain" im Namen
        - Grund: FlixTrain ist Schienenverkehr und bereits im öV-Netz erfasst
    - **Räumliche Duplikate:** Entfernt Haltestellen < 100m Distanz (Haversine-Formel)
        - Grund: Flixbus/BlaBlaCar nutzen oft dieselben Haltestellen mit leicht unterschiedlichen Koordinaten
    - **ÖV-Duplikate:** Vergleich mit BFKOORD_WGS um bereits erfasste Haltestellen zu filtern
7. **ID-Vergabe:** Vergibt fortlaufende IDs ab 1700000 an bereinigte Daten
    - Zweck: Eindeutige Identifikation für öV-Systeme (siehe "ID-Vergabe" oben)

- Wichtig: IDs werden NACH Bereinigung vergeben um Lücken zu vermeiden

8. **CSV-Export**: Erstellt kommagetrennte *csv*-Dateien für beide BFKOORD_WGS-Dateien
    - Zweck: Einfacher Import in QGIS
9. **BAHNHOF-Format**: Konvertiert zu Legacy-Format für öV-Systeme
    - Zweck: Kompatibilität mit bestehenden öV-Import-Systemen
10. **METABHF-Erstellung**: Automatische oder manuelle Generierung
    - **Automatisch (OSM)**: Verwendet OSMnx für Walking-Distance-Berechnung zwischen Coach- und öV-Haltestellen
        - Algorithmus: Findet nächste 3 öV-Haltestellen, berechnet Gehweg über Straßennetz, wählt kürzeste Route
        - Distanz → Gehzeit (5 km/h) + 2 Minuten Puffer
    - **Manuell (QGIS)**: Räumliche Analyse in QGIS für präzise Umsteigebeziehungen
        - Workflow: QGIS → *csv* Export → `process_qgis_metabhf.py` → delta/METABHF
11. **UMSTEIGB-Generierung**: Erstellt Umsteigezeiten aus BAHNHOF + METABHF
    - Format: Siehe oben
    - Zweck: Definition von Umsteigezeiten zwischen Haltestellen
12. **Delta-ZIP**: Optional: Packt alle Delta-Dateien für Weitergabe
    - Enthält: BFKOORD_WGS, BAHNHOF, METABHF, UMSTEIGB

METABHF-Verarbeitung

Die Pipeline bietet zwei Workflows für METABHF-Erstellung:

Option 1: Automatisch via OSM (Empfohlen)

Wird automatisch in `main.py` ausgeführt. Verwendet OpenStreetMap-Daten für Walking-Distance-Berechnung.

Option 2: Manuell via QGIS

⚠️ **Hinweis**: Manuelle QGIS-Verarbeitung erfolgt als separater Workflow:

1. Führe die ETL-Pipeline aus: `python main.py`
2. Öffne QGIS und führe räumliche Analyse durch
3. Exportiere METABHF als *csv* nach `data/processed/QGIS_METABHF.csv`
4. Führe Post-Processing aus: `python src/transfer_stops/create_metabhf/process_qgis_metabhf.py`

a. ⚠️ Achtung: Überschreibt existierende `delta/METABHF` und `delta/UMSTEIGB` Dateien

Das Skript:

- Liest QGIS *csv* und entfernt Sonderzeichen
- Kopiert Original-Format (ID1 ID2 000 / *A Y)
- Gruppiert nach Ziel-ID: `ID2 : ID1a, ID1b, ...`
- Erstellt automatisch UMSTEIGB (überschreibt vorhandene Datei)
- Optional: Zipped Delta-Dateien

Ausgabedateien

Generierte Dateien

- `data/processed/delta/BFKOORD_WGS`: Neue Haltestellen mit Koordinaten und IDs
- `data/processed/delta/BAHNHOF`: BAHNHOF-Format für öV-Systeme
- `data/processed/delta/METABHF`: Umsteigebeziehungen (OSM oder QGIS)
- `data/processed/delta/UMSTEIGB`: Umsteigezeiten (automatisch generiert)
- `data/processed/BFKOORD_WGS_KOMMAGETRENNT.csv`: *csv* der neuen Haltestellen zum Einladen in QGIS
- `data/processed/OEV_BFKOORD_WGS_KOMMAGETRENNT.csv`: öV-Referenz-Koordinaten als *csv* zum Einladen in QGIS
- `data/processed/provider_stops`: Konsolidierte Schweizer Haltestellen aller Provider
- `data/processed/delta.zip`: Gezippte Delta-Dateien (BFKOORD_WGS, BAHNHOF, METABHF, UMSTEIGB)

Konfiguration

`src/transfer_stops/config.py`:

- `BEGINNING_ID`: Start-ID für neue Haltestellen (Standard: 1700000)
- `OEV_SAMMLUNG_URL`: Permalink zu öV-Referenzdaten
- `oev_files`: Liste der öV-Dateien (BAHNHOF, BFKOORD_WGS, METABHF, UMSTEIGB)
- `PATHS`: Dictionary mit allen Dateipfaden (zentrale Pfad-Verwaltung)
- `providers`: Liste der Transport-Provider mit GTFS-URLs und Konfiguration

Pfad-Verwaltung: Alle Dateipfade sind zentral in `config.PATHS` definiert, um hardcodierte Pfade zu vermeiden.

Clean-Up

```
python src/transfer_stops/clean_data.py
```

Löscht alle heruntergeladenen und generierten Daten für einen kompletten Neustart der Pipeline.

6 Methoden zur Berechnung der Umsteigebeziehungen

Für diese Arbeit werden vier verschiedene Methoden betrachtet um die Umsteigebeziehung zu berechnen. Drei davon werden in QGIS umgesetzt, während eine Methode direkt in Python entwickelt wird. Ziel ist es, die Berechnungsansätze miteinander zu vergleichen, welche idealerweise zu ähnlichen Ergebnissen führen. Nach der Durchführung aller Methoden, werden die berechneten Zeiten anhand aktueller Geodaten von Google Maps miteinander verglichen. Um die Analysen in den Testkapiteln zu vereinfachen, wird für jede Methode eine Vergleichstabelle erstellt und mittels einem R-Skript in RStudio (siehe Anhang D) mehrere Grafiken erzeugt, welche die Unterschiede zwischen den beiden Methoden anschaulich darstellen.

Durch diesen Vergleich lässt sich die Validität der Resultate überprüfen. Als Ergebnis wird jene Methode ausgewählt, welche sowohl verlässliche Ergebnisse liefert, als auch in der praktischen Umsetzung am verständlichsten und effizientesten ist.

Vom Auftraggeber wird definiert, dass je berechneter Umsteigezeit zusätzlich 2 Minuten dazugerechnet werden sollen, sodass das Be- und Entladen allfälliger Gepäckstücke miteinbezogen werden kann. Beim Erstellen der `METABHF`-Datei wird diese berücksichtigt, bei der Analyse bzw. dem Vergleich mit Google Maps, wird diese wiederrum abgezogen.

6.1 Netzwerk-Analyse

Als erste Methode wird eine Netzwerk-Analyse in QGIS angewandt. Anhand dieser wird der kürzeste Weg oder die benötigte Zeit einer Strecke ermittelt. Dabei werden die kumulativen Kosten zwischen zwei Knotenpunkten innerhalb eines Netzwerks berechnet. Im Gegensatz zur Luftlinie werden die Kosten entlang des tatsächlichen Weges im Netzwerk aufsummiert. Je nach Einstellung entsprechen diese Kosten entweder der Länge in Metern oder der Zeit in Sekunden oder Minuten (dges.carleton.ca, 2021).

QGIS durchläuft das Netzwerk dabei abschnittsweise, also Strassenabschnitt für Strassenabschnitt (Kante für Kante), und addiert die Kosten jedes einzelnen Abschnitts. Auf diese Weise wird der gesamte Weg vom Startpunkt bis zum Endpunkt berechnet. Als Knoten gelten Kreuzungen sowie Endpunkte von Linien. Die Verbindung zwischen zwei Konten wird durch eine Kante definiert, welche Strassen oder Wege darstellen. Innerhalb der Netzwerk-Analyse kann zwischen dem kürzesten Weg, bei dem für alle Strassen dieselbe Geschwindigkeit angenommen wird, und dem schnellsten Weg, bei dem unterschieliche Geschwindigkeiten berücksichtigt werden, unterschieden werden. Für die vorliegende Analyse ist der schnellste Weg die massgebende Variante (dges.carleton.ca, 2021).

In dieser Methode, wie in allen nachfolgenden Methoden auch, werden die beiden `BFKOORD_WGS`-Dateien verwendet: Eine mit den Haltestellen der Fernverkehrsunternehmen und eine mit den öV-Haltestellen aus *opentransportdata.swiss*. Es ist darauf zu achten, dass beide Dateien eine ID und dazugehörige Koordinaten enthalten. Des Weiteren müssen sich die IDs der beiden Datensätze voneinander unterscheiden. Da der ETL-Prozess aus Abschnitt 5 diese Anforderungen bereits sicherstellt, sind hier keine zusätzlichen Anpassungen notwendig.

Für die Berechnung der Gehzeiten wird ein Fusswegnetz aus OpenStreetMap (OSM) verwendet. Dazu wird das Strassennetz ins QGIS geladen und für die Analyse vorbereitet, sodass zusammenhängende und korrekt verbundene Wege vorhanden sind.

Um zu vermeiden, dass für jeden Startpunkt alle möglichen Zielhaltestellen im gesamten Datensatz berücksichtigt werden müssen, wird um die Fernverkehrshaltestellen ein räumlicher Puffer von

1500 Metern gelegt. Dadurch bleiben nur die nächstgelegenen öV-Haltestellen für die weitere Analyse erhalten.

Im Anschluss sollte der kürzeste, beziehungsweise schnellste Weg, mit dem QGIS-Werkzeug „Kürzester Weg (Punkt zu Layer)“ berechnet werden. Bei der praktischen Anwendung zeigte sich jedoch, dass dieses Werkzeug immer nur für einen einzelnen Startpunkt ausgeführt werden kann. Zwar können mehrere Zielpunkte angegeben werden, der Startpunkt muss jedoch jeweils manuell gesetzt werden.

Da rund 30 Startpunkte vorhanden sind, wäre eine manuelle Durchführung grundsätzlich möglich. Allerdings benötigt der Algorithmus pro Startpunkt etwa zwei Stunden Rechenzeit, was insgesamt zu einem sehr hohen Zeitaufwand führt. Aus diesem Grund wird diese Methode nicht weiterverfolgt.

6.2 QNEAT3 Plugin

Die zweite Methode nutzt das QNEAT3-Plugin. Dabei handelt es sich um ein kostenloses Zusatzmodul für QGIS. QNEAT3 ist in der Programmiersprache Python geschrieben und direkt in das Verarbeitungswerkzeug von QGIS integriert. Das Plugin stellt verschiedene Werkzeuge für Netzwerkanalysen zur Verfügung. Diese reichen von einfachen Berechnungen des kürzesten Weges bis hin zu weiterführenden Analysen wie Isochronen-Flächen oder der Berechnung von Origin-Destination-Matrizen. Damit können Distanzen und Erreichbarkeiten auf Basis eines realen Wegenetzes ermittelt werden (Raffler, 2018a).

Der zur Berechnung des kürzesten Weges verwendete Algorithmus basiert auf dem Standard-Netzwerkanalyse-Werkzeug von QGIS, welches ursprünglich von Alexander Bruy entwickelt wurde. Bei der Berechnung des kürzesten Weges berücksichtigt QNEAT3 nicht nur das eigentliche Strassennetz, sondern einerseits auch die kurzen Wege vom Startpunkt zum nächstgelegenen Punkt auf dem Netzwerk und anderseits auch die vom Netzwerk zum Zielpunkt. Dadurch kann der tatsächliche Weg zwischen zwei Haltestellen realistischer abgebildet werden, auch wenn dieser nicht direkt auf einer Strasse oder einem Gehweg liegt (Raffler, 2018b).

QNEAT3 führt ebenfalls Netzwerk-Analysen durch und bringt auch hier kein eigenes Strassennetz mit. Das benötigte Netzwerk muss deshalb zusätzlich als Layer in QGIS geladen werden. Für diese Analyse wurde das swissTLM3D-Datenset von swisstopo verwendet, welche ein detailliertes und topologisch sauberes Strassen- und Wegenetz für die Schweiz enthält (swisstopo.admin, 2024).

6.2.1 Anwendungsbeispiel

Nach dem Einladen der Fernverkehrshaltestellen als *csv* sowie des öV-Haltestellen-Layers, wird das Strassennetz aus dem swissTLM3D-Datensatz als weiterer Layer in QGIS hinzugefügt. Dieses Netz dient QNEAT3 als Grundlage für die Berechnung der Gehwege.

Gleich wie bei der in Kapitel 6.1 beschriebenen Netzwerk Analyse, kann auch QNEAT3 nicht alle Verbindungen zwischen den Start- und Ziel-Haltestellen gleichzeitig verarbeiten, da das Plugin nur eine begrenzte Anzahl von Punkten effizient bearbeiten kann. Um dieses Problem zu umgehen, wird auch hier ein Puffer von 1500 Metern um die Fernverkehrshaltestellen gelegt. Dadurch werden nur jene öV-Haltestellen ausgewählt, die sich im direkten Umfeld der Fernverkehrshaltestellen befinden. Auf diese Weise wird die Anzahl der zu berücksichtigenden Zielpunkte stark reduziert.

Statt alle möglichen Kombinationen zwischen rund 30 Fernverkehrshaltestellen und mehreren zehntausend öV-Haltestellen zu berechnen, beschränkte sich die Analyse auf eine deutlich kleinere und realistische Teilmenge der nächstgelegenen Haltestellen. Dadurch wird die Berechnung mit QNEAT3 stark beschleunigt.

6.2.2 Analyse

Die Ergebnisse der mit dem QNEAT3-Plugin berechneten Gehzeiten wird mit den entsprechenden Gehzeiten aus Google Maps verglichen. Die vollständige Vergleichstabelle mit allen Start- und Zielkombinationen ist im Anhang A beigefügt.

Die erste Grafik zeigt die Gehminuten von QNEAT3 und Google Maps für jede Strecke nebeneinander. Die Routen sind dabei nach den QNEAT3-Gehzeiten aufsteigend sortiert, sodass kurze und lange Wege miteinander verglichen werden können.

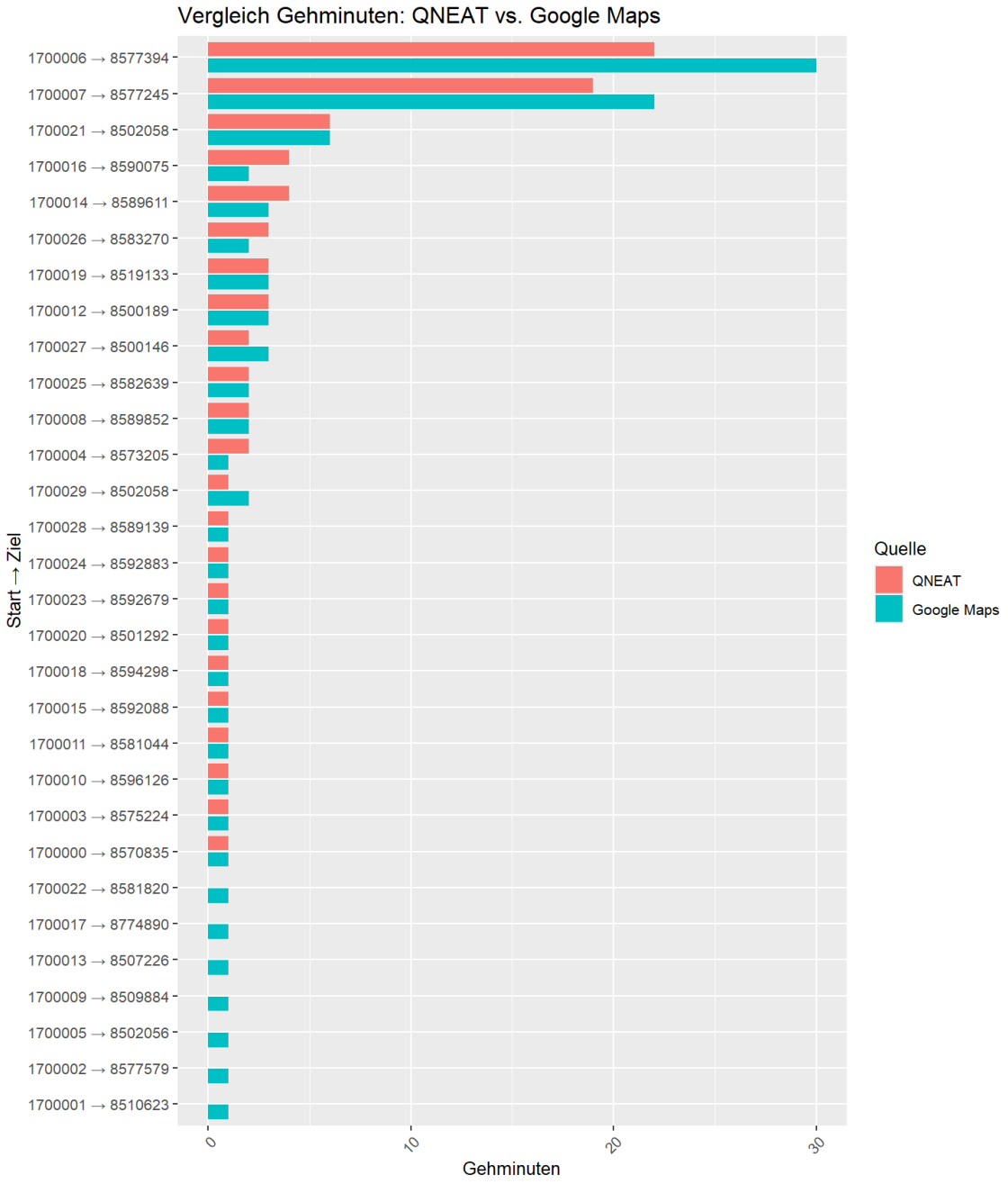


Abbildung 4: Vergleich Gehminuten: QNEAT3 vs. Google Maps

Dabei ist zu beachten, dass Google Maps selbst bei Strecken mit einer Distanz von 0 Metern standardmässig eine Gehzeit von einer Minute angibt. Dieser Umstand führt insbesondere bei sehr kurzen Verbindungen zu systematischen Abweichungen zwischen den beiden Methoden.

Die zweite Grafik stellt die Differenz der Gehzeiten (QNEAT3 minus Google Maps) in Minuten dar. Positive Werte bedeuten, dass QNEAT3 längere Gehzeiten berechnet als Google Maps, Negative entsprechend kürzere. Die Sortierung der Balken ermöglicht es, Ausreisser sowie typische Abweichungen auf einen Blick zu erkennen.



Abbildung 5: Delta (QNEAT3 - Google Maps)

Aus den Ergebnissen ergeben sich die folgenden Abweichungen:

Tabelle 2: Verteilung der absoluten Abweichungen von QNEAT3

Abweichung	**Anzahl**	**Anteil**
≤ 1 Minute	27	90,0 %
2–3 Minuten	2	6,7 %
≥ 4 Minuten	1	3,3 %
Summe	**30**	**100 %**

Tabelle 3: Zusammenfassung der Abweichungen (in Minuten) von QNEAT3

Kennzahl	**Wert**
Durchschnittliche Abweichung	0,83 Minuten
Median	0,5 Minuten
Grösste Abweichung	8 Minuten

Insgesamt passen die beiden Gehzeit-Werte meistens sehr gut überein. Bei 90 % der Routen liegt der Unterschied bei höchstens einer Minute. 2 Routen (6,7 %) weichen um 2–3 Minuten ab und eine Route (3,3 %) zeigt eine Differenz von mehr als 4 Minuten.

Über alle Routen hinweg beträgt die Abweichung im Schnitt 0,83 Minuten (weniger als eine Minute). Der Median liegt bei 0,5 Minuten, was zeigt, dass die meisten Differenzen sogar kleiner als der Durchschnitt sind. Der grösste Ausreisser liegt bei 8 Minuten.

6.3 ORS Tools Plugin

Als dritte Methode für die Berechnung der tatsächlichen Gehzeiten wird das QGIS-Plugin ORS Tools verwendet, welches auf dem Dienst von OpenRouteService (ORS) basiert. ORS ist ein weltweit verfügbarer Routing-Dienst, der die Berechnung von Routen sowie Navigationsinformationen ermöglicht. Dabei können unterschiedliche Fortbewegungsarten wie Auto, Fahrrad, Fussverkehr, Wandern, Rollstuhl oder Lkw ausgewählt werden. Für jede dieser Nutzergruppen verwendet ORS ein speziell auf das jeweilige Profil abgestimmtes Strassennetz, um realistische und nutzerspezifische Routen zu berechnen (Services | openrouteservice, o. J.).

Das ORS-Tools-Plugin ist in der Lage, komplexe Routingaufgaben direkt in QGIS auszuführen und greift dabei, wie Methode eins, auf Daten aus OSM zurück (*Plugins QGIS*, o. J.). Durch die Nutzung von OSM steht ein globales, frei verfügbares Strassen- und Fusswegnetz zur Verfügung, welches als Grundlage für die Routenberechnung dient (Gandhi, 2025).

Um das ORS Tools Plugin anwenden zu können, wird ein API Key benötigt. Dafür muss der Benutzer sich auf https://openrouteservice.org/sign-up/ registrieren. Dieser steht kostenlos zur Verfügung.

6.3.1 Anwendungsbeispiel

Da diese Methode ein eigenes Strassennetz benutzt, kann zur besseren Orientierung zusätzlich eine Schweizer Hintergrundkarte (z. B. swisstopo) in QGIS geladen werden. Diese dient ausschliesslich der visuellen Veranschaulichung und hat keinen Einfluss auf die Berechnung der Gehwege.

Ähnlich wie bei Methode 2 besteht auch hier das Problem, dass eine direkte Berechnung zwischen allen Fernverkehrs- und öV-Haltestellen zu viele Kombinationen erzeugen würde. Um die Anzahl der Zielhaltestellen zu reduzieren, wird auch in dieser Methode die Anzahl der möglichen Zielhaltestellen eingeschränkt, indem je Fernverkehrshaltestelle nur die vier nächstgelegenen öV-Haltestellen ausgewählt werden. Allfällige Duplikate, die durch diese Auswahl entstehen können, werden anschliessend entfernt, sodass je Fernverkehrshaltestelle eine klar definierte Menge potenzieller Umsteigepunkte verbleibt.

Auf Basis dieser reduzierten Punktmenge wird anschliessend mit ORS Tools eine Distanz- und Zeitmatrix berechnet. Als Verkehrsmodus wird dabei „foot-walking“ gewählt, sodass die Gehzeiten entlang des realen Fusswegnetzes ermittelt werden. Aus allen berechneten Verbindungen wird je Fernverkehrshaltestelle diejenige mit der kürzesten Gehzeit ausgewählt.

6.3.2 Analyse

Wie bereits bei der QNEAT3 Methode in Kapitel 6.2 werden auch hier die Abweichungen der berechneten Gehzeiten zu Google Maps anhand einer Vergleichstabelle (Anhang B) näher betrachtet. Zunächst erfolgt dies anhand einer visuellen Darstellung der Gehminuten, bei der die von ORS Tools berechneten Zeiten, den Referenzwerten von Google Maps gegenübergestellt werden. Die Routen sind dabei auch hier nach den ORS-Tools-Gehzeiten sortiert, wodurch Unterschiede zwischen kurzen und längeren Strecken gut erkennbar sind.

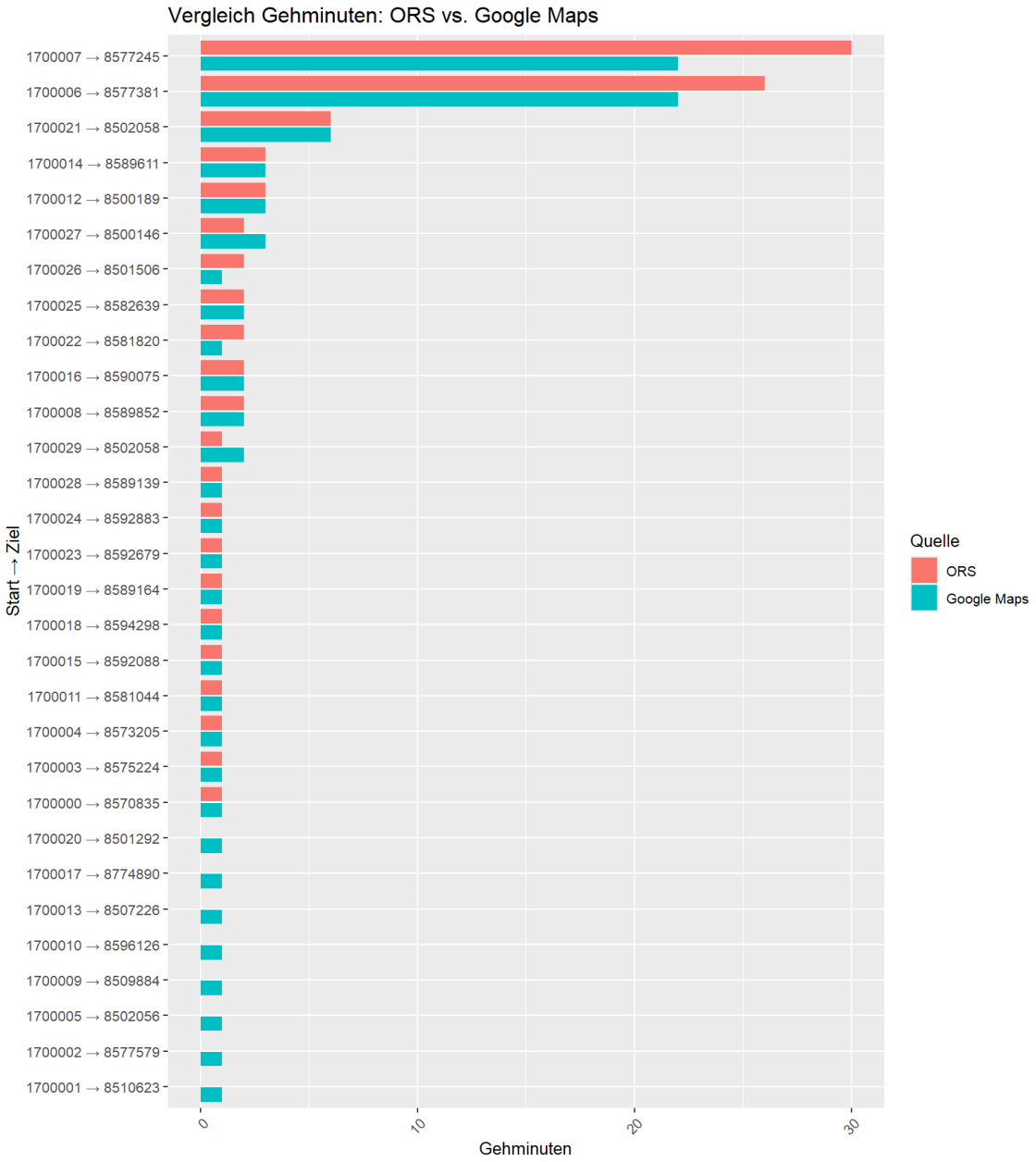


Abbildung 6: Vergleich Gehminuten: ORS Tools vs. Google Maps

Anschliessend wird die Differenz der Gehzeiten als Minuten-Delta dargestellt. Diese Darstellung zeigt für jede Verbindung, ob ORS Tools kürzere oder längere Gehzeiten berechnet als Google Maps und macht sowohl typische Abweichungen, als auch einzelne Ausreisser sichtbar.

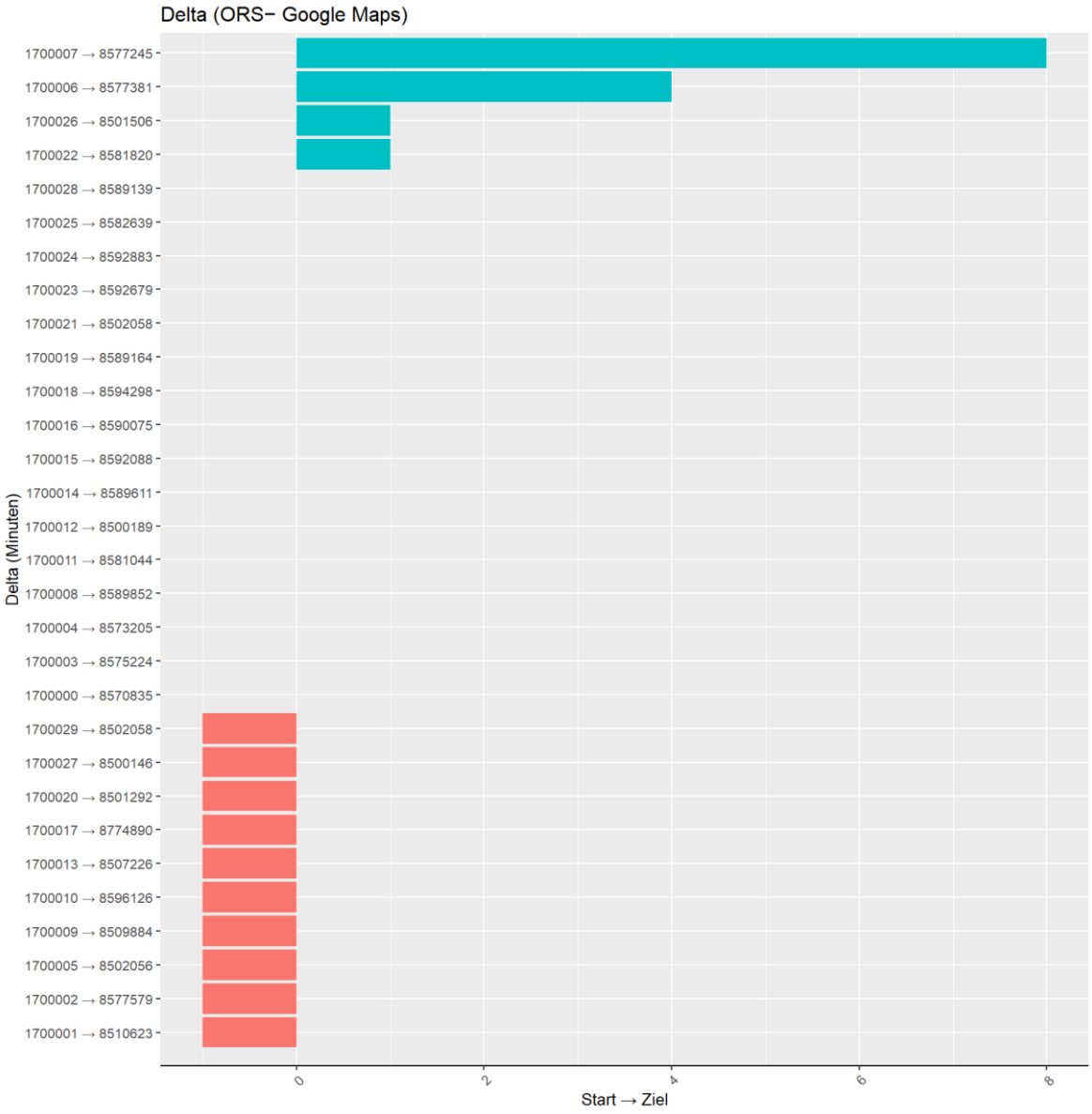


Abbildung 7: Delta (ORS Tools - Google Maps)

Auf Basis dieser Ergebnisse lassen sich folgende Kennzahlen berechnen:

Tabelle 4: Verteilung der absoluten Abweichungen von ORS Tools

Abweichungsklasse	**Anzahl**	**Anteil (%)**
≤ 1 Minute	28	93,3 %
2–3 Minuten	0	0 %
≥ 4 Minuten	2	6,7 %
Summe	**30**	**100 %**

Tabelle 5: Zusammenfassung der Abweichungen (in Minuten) von ORS Tools

Kennzahl	**Wert**
Durchschnittliche Abweichung	0,8 Minuten
Median der Abweichung	0 Minuten
Maximale Abweichung	8 Minuten

Der Vergleich der mit ORS Tools berechneten Gehzeiten mit den Referenzwerten von Google Maps zeigt insgesamt eine sehr hohe Übereinstimmung. In 93,3 % der untersuchten Fälle liegt die Abweichung bei höchstens einer Minute, was darauf hinweist, dass ORS Tools für die meisten Relationen sehr ähnliche Gehzeiten liefert wie Google Maps. Lediglich zwei Routen (6,7 %) weisen grössere Abweichungen von vier Minuten oder mehr auf.

Über alle Relationen hinweg beträgt die durchschnittliche absolute Abweichung 0,8 Minuten, bei einem Median von 0 Minuten. Der grösste beobachtete Unterschied liegt bei 8 Minuten und stellt damit einen klaren Ausreisser dar.

6.4 OSMnx anhand Python

Um die Suche nach passenden Umsteigebeziehungen für den Auftraggeber zu vereinfachen, wurde zusätzlich zum QGIS-Workflow ein eigener Python-Code entwickelt.

Dieser Code verwendet dieselbe Grundidee wie die dritte Methode in QGIS, welche zur Berechnung der Gehwegezeiten zwischen Fernverkehrshaltestellen und bestehenden öV-Haltestellen das Strassennetz aus OSM nutzt. Dabei lädt das Python-Script mit Hilfe der Bibliothek OSMnx das Strassen- und Fusswegnetz von OSM herunter, modelliert es als Graph und berechnet auf diesem Netzwerk die kürzesten Wege automatisch (Boeing, 2025).

6.4.1 Anwendungsbeispiel

Der Python-Code liest im ersten Schritt die beiden `BFKOORD_WGS`-Dateien ein. Für jede Fernverkehrshaltestelle werden danach die nächsten öV-Haltestellen über die Luftlinie bestimmt. Aus dieser Liste werden auch hier die vier nächstgelegenen Standorte ausgewählt. Anschliessend wird mit OSMnx rund um die Fernverkehrshaltestelle ein Fusswegnetz aus OSM geladen. Aus diesem Netzwerk berechnet das Skript mit NetworkX den kürzesten begehbaren Weg zu den ausgewählten öV-Haltestellen.

Für jede dieser möglichen Verbindungen wird der tatsächliche Gehweg in Metern und die entsprechende Gehzeit in Minuten berechnet. Die Gehzeit basiert auf der im Python Projekt selber definierten Geschwindigkeit von 5 km/h. Von allen berechneten Kandidaten wird jeweils die Verbindung mit der kürzesten Gehzeit ausgewählt.

Die so ermittelten Umsteigebeziehungen werden am Ende automatisch in einer Datei im `ME-TABHF`-Format gespeichert. Dabei wird für jede Fernverkehrshaltestelle festgehalten, zu welcher öV-Haltestelle eine Umsteigebeziehung besteht und wie lange der Fussweg dauert. Im zweiten Teil der Datei werden alle Fernverkehrshaltestellen je öV-Haltestelle gruppiert, sodass auf einen Blick ersichtlich ist, welche Fernverkehrshaltestellen an eine bestimmte öV-Haltestelle angebunden sind.

Durch diesen Ansatz können die Umsteigebeziehungen ohne zusätzliche manuelle Schritte in QGIS berechnet werden. Demnach muss in dieser Methode lediglich die Pipeline gestartet werden,

woraufhin der Code eine aktuelle `METABHF`-Datei auf Basis der neuesten Haltestellendaten und der OSM-Fusswege automatisch erzeugt.

6.4.2 Analyse

Auch bei dieser Methode werden anhand einer Vergleichstabelle (Anhang C) die berechneten Gehzeiten mit den Referenzwerten von Google Maps verglichen. Zunächst erfolgt dies anhand einer visuellen Darstellung der Gehminuten, bei der die mit OSMnx berechneten Zeiten den Gehzeiten von Google Maps gegenübergestellt werden. Die Verbindungen sind dabei nach den OSMnx-Gehzeiten sortiert, wodurch Unterschiede zwischen den einzelnen Relationen sichtbar werden.

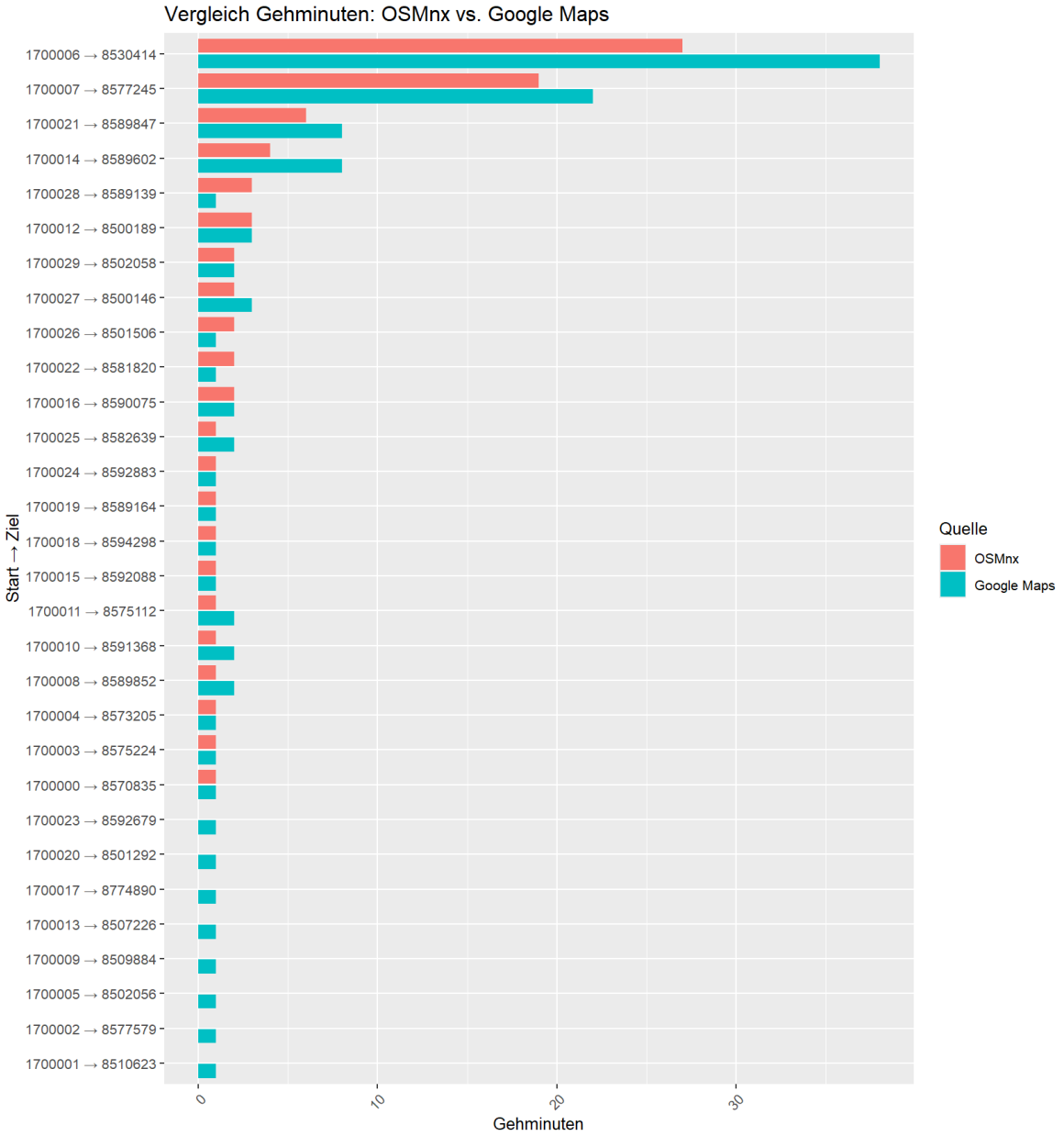


Abbildung 8: Vergleich Gehminuten: OSMnx vs. Google Maps

Anschliessend wird die Differenz der Gehzeiten als Minuten-Delta dargestellt. Diese Darstellung zeigt, ob OSMnx für eine Verbindung kürzere oder längere Gehzeiten berechnet als Google Maps und macht sowohl typische Abweichungen als auch einzelne Ausreisser erkennbar.

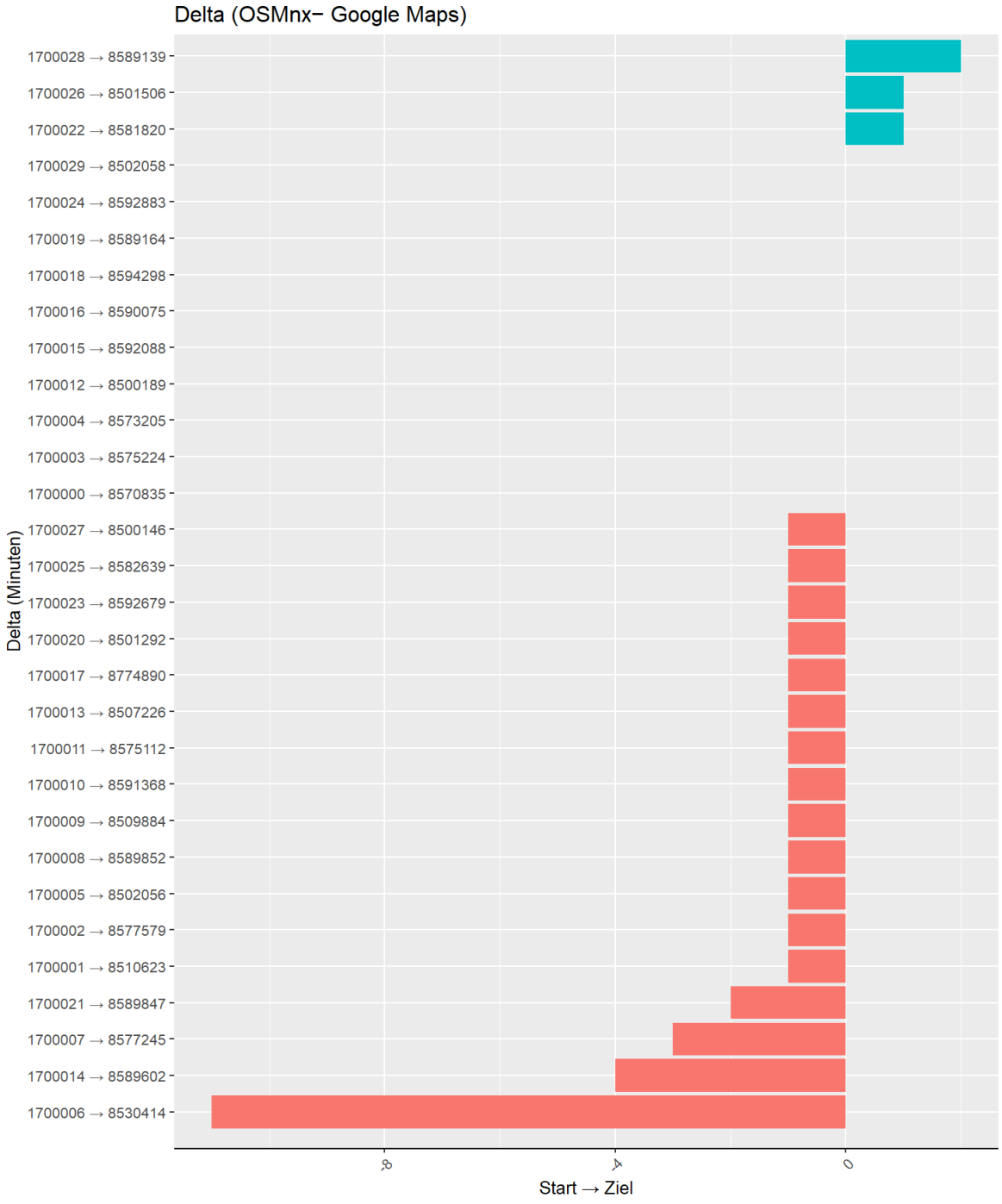


Abbildung 9: Delta (OSMnx - Google Maps)

Auf Basis dieser Ergebnisse ergeben sich folgende Kennzahlen:

Tabelle 6: Verteilung der absoluten Abweichungen von OSMnx

Abweichung	**Anzahl**	**Anteil**
≤ 1 Minute	25	83,3 %
2–3 Minuten	3	10,0 %
≥ 4 Minuten	2	6,7 %
Summe	**30**	**100 %**

Tabelle 7: Zusammenfassung der Abweichungen (in Minuten) von OSMnx

Kennzahl	**Wert**
Durchschnittliche Abweichung	1,23 Minuten
Median	1 Minute
Grösste Abweichung	11 Minuten

Der Vergleich der Gehzeiten zeigt, dass die berechneten Werte in den meisten Fällen sehr nahe beieinanderliegen. Bei 83,3 % der Routen beträgt der Unterschied weniger als eine Minute, was auf eine insgesamt gute Übereinstimmung hinweist. In 10 % der Fälle liegt die Abweichung bei zwei bis drei Minuten, während nur wenige Routen (6,7 %) deutlich grössere Unterschiede von vier Minuten oder mehr aufweisen.

Insgesamt ergibt sich eine durchschnittliche Abweichung von 1,23 Minuten. Der Median liegt bei 1,0 Minuten, was zeigt, dass grössere Abweichungen Ausnahmen sind. Der grösste Unterschied beträgt 11 Minuten und stellt einen klaren Ausreisser dar.

6.5 Methodenvergleich

Im Methodenvergleich wird die Netzwerkanalyse nicht berücksichtigt, da dafür keine verwertbaren Ergebnisse vorliegen. Da sowohl die QNEAT3-Methode als auch ORS Tools in QGIS durchgeführt wurden und ORS Tools dabei die besseren Ergebnisse geliefert hat, wird für den weiteren Vergleich nur ORS Tools und OSMnx herangezogen. Ein weiterer Grund für diesen Vergleich ist, dass sowohl ORS Tools als auch die Python-basierte Analyse mit OSMnx auf Daten aus OSM basieren. Dadurch lassen sich die Resultate besser miteinander vergleichen und Unterschiede gezielt untersuchen.

Nach der Auswertung der ermittelten Zielhaltestellen zeigte sich, dass sich bei fünf Haltestellen die ermittelten Zielhaltestellen zwischen der ORS-Tools-Methode und der OSMnx-Methode unterscheiden. Diese Fälle werden im Folgenden genauer analysiert und sind in der nachfolgenden Tabelle aufgeführt.

Tabelle 8: Zeitliche Unterschiede zwischen OSMnx, ORS Tools und Google Maps

Start → Ziel (OSMnx)	OSMnx (min)	Google Maps (min)	Start → Ziel (ORS Tools)	ORS Tools (min)	Google Maps (min)
1700006→8530414	27	38	1700006→8577381	26	22
1700010→8591368	1	2	1700010→8596126	0	1
1700011→8575112	1	2	1700011→8581044	1	1
1700014→8589602	4	8	1700014→8589611	3	3
1700021→8589847	6	8	1700021→8502058	6	6

Anhand der folgenden Abbildungen wird die nächstgelegene Haltestelle dargestellt.

1700006 Erstfeld Gotthard Service Station South

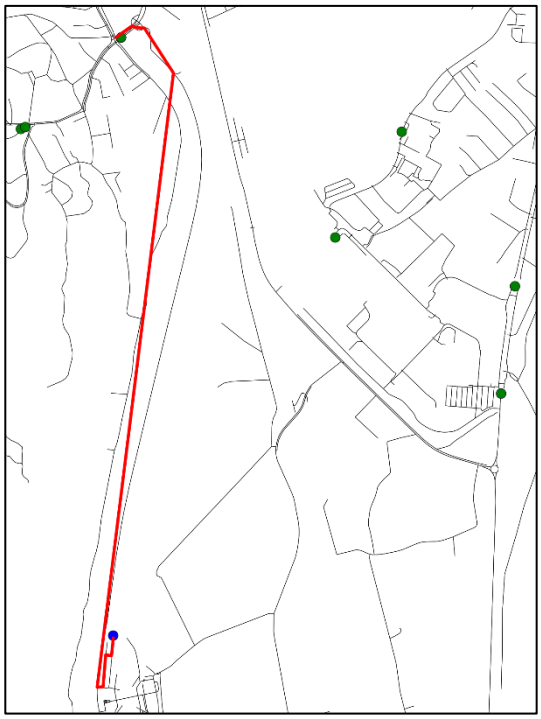


Abbildung 10: OSMnx Strecke Erstfeld Gotthard Service Station South

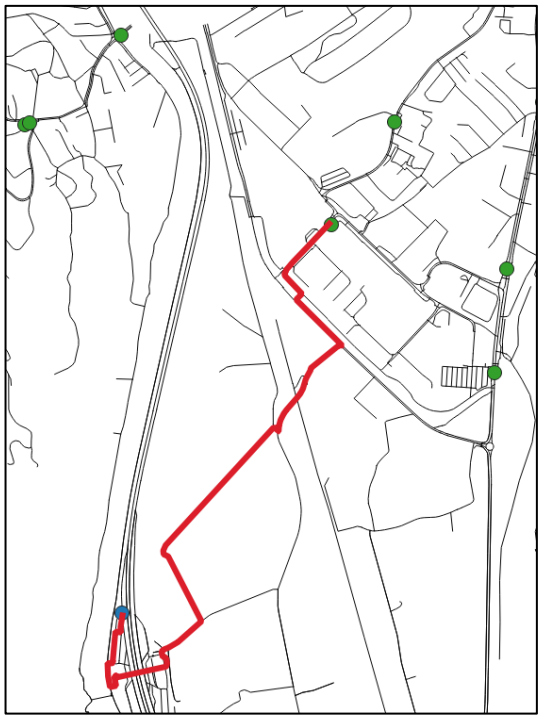


Abbildung 11: ORS Strecke Erstfeld Gotthard Service Station South

1700010 Zurich Bus Station

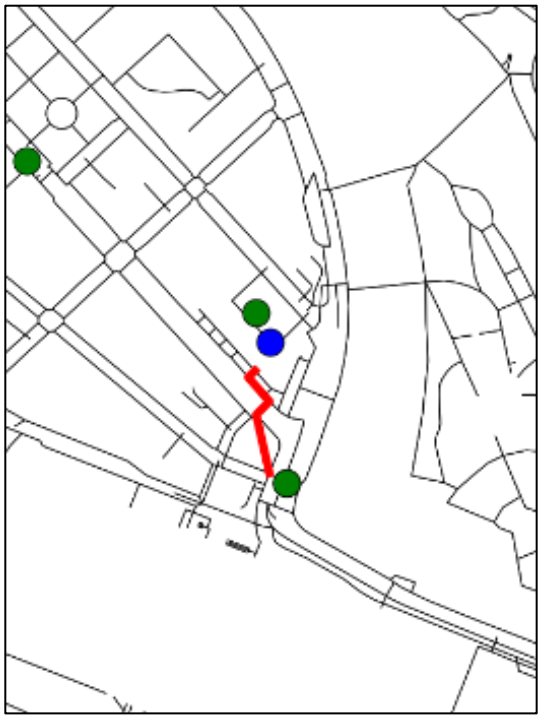


Abbildung 12: OSMnx Strecke Zurich Bus Station

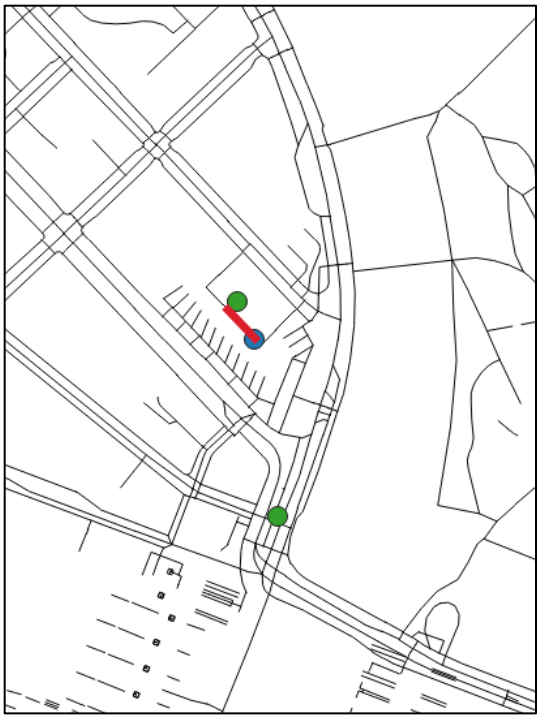


Abbildung 13: ORS Tools Strecke Zurich Bus Station

1700011 Chur station train station (Bahnhofplatz)

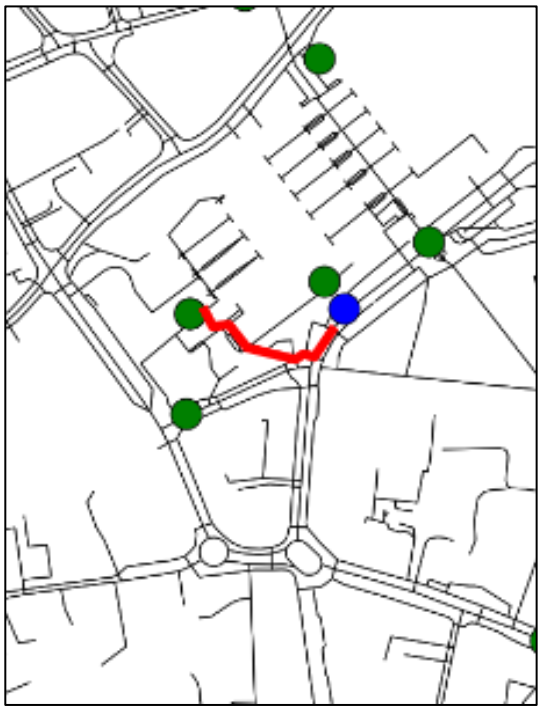


Abbildung 14: OSMnx Strecke Chur station train station (Bahnhofplatz)

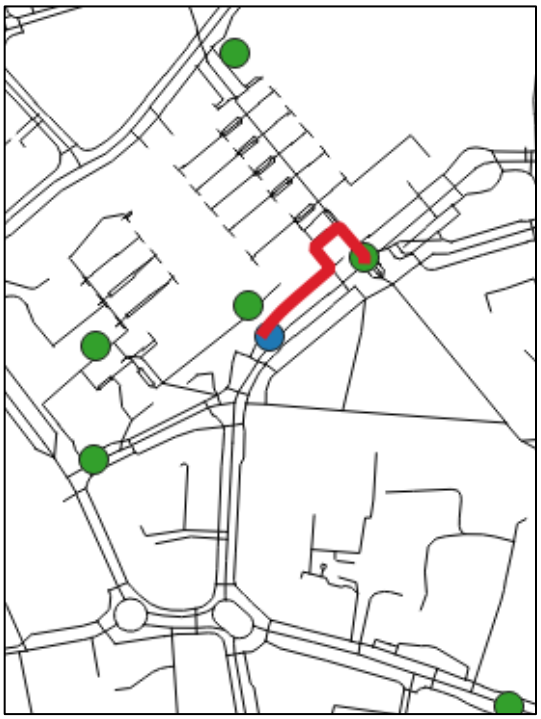


Abbildung 15: ORS Tools Strecke Chur station train station (Bahnhofplatz)

1700014 St. Gallen train station

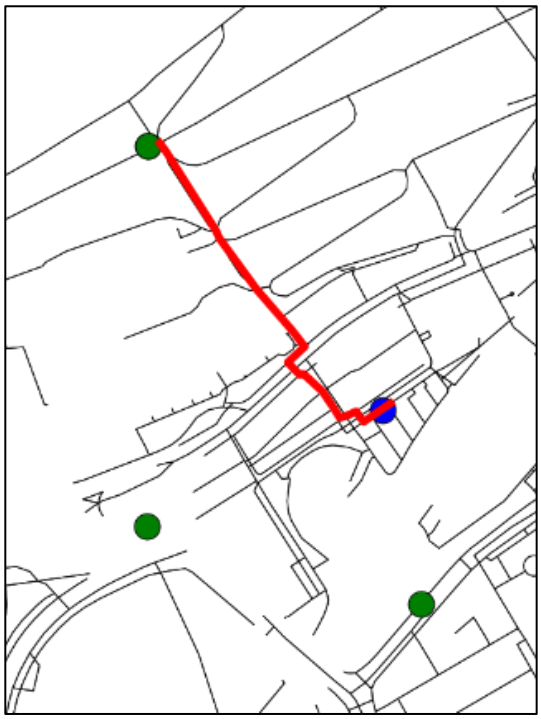


Abbildung 16: OSMnx Strecke St. Gallen train station



Abbildung 17: ORS Tools Strecke St. Gallen train station

1700021 Luzern (Landenberg)

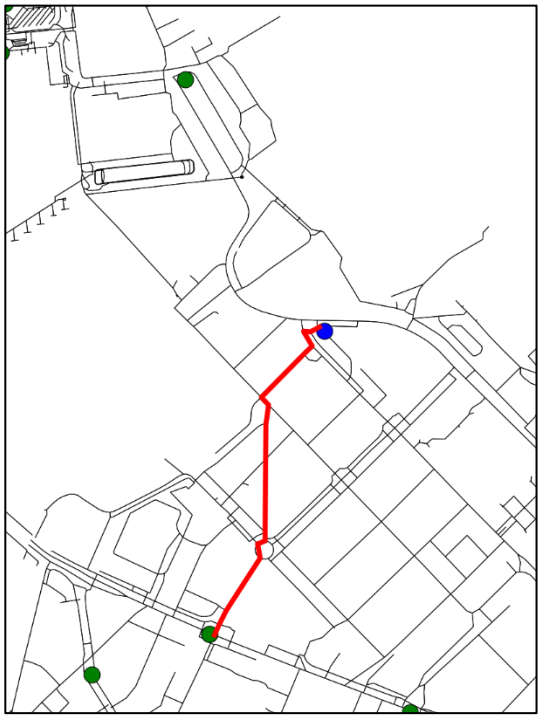


Abbildung 18: OSMnx Strecke Luzern (Landenberg)

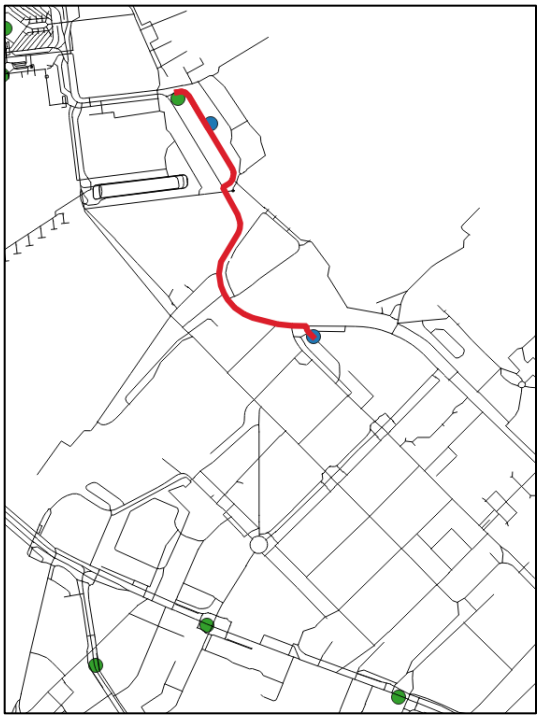


Abbildung 19: ORS Tools Strecke Luzern (Landenberg)

Allgemein zeigt sich, dass der zeitliche Unterschied zwischen den unterschiedlichen Zielhaltestellen nur zwischen 0 und 1 Minute liegt.

Der Unterschied bei der Fernverkehrshaltestelle mit der ID 170006 lässt sich dadurch erklären, dass in QGIS nur die vier nächstgelegenen Haltestellen nach Luftlinie berücksichtigt werden. Dadurch kann es vorkommen, dass die tatsächlich nächstgelegene Haltestelle gemäss OSMnx in den ORS Tools gar nicht zur Auswahl steht.

Bei den IDs 1700010, 1700011 und 1700021 liegt der Unterschied darin, dass die berechneten Routen (rote Linien) nicht exakt am gleichen Ort beginnen und enden. Obwohl Start- und Zielpunkte identisch sind, starten und enden die Routen je nach Methode leicht versetzt auf dem Wegenetz. Dadurch entstehen Abweichungen von wenigen Metern, welche sich direkt auf die berechneten Gehminuten auswirken.

Bei der ID 1700014 liegt der Unterschied darin, dass in OSMnx gewisse Strassen fehlen, welche in den ORS Tools vorhanden sind. Dadurch wird dort eine andere Route berechnet.

Der Methodenvergleich zeigt, dass bei insgesamt 30 untersuchten Fernverkehrshaltestellen lediglich bei fünf Haltestellen ein unterschiedliches Ziel ermittelt wurde. Die dabei entstehenden Unterschiede in den Gehminuten sind gering und bewegen sich meist im Bereich von weniger als 1 Minute. Für die Auswahl einer geeigneten Methode spielt daher weniger das Ergebnis selbst, sondern vielmehr die praktische Anwendbarkeit eine zentrale Rolle.

Um eine zukünftige Nutzung von ORS Tools durch den Auftraggeber zu ermöglichen, wurde im Anhang (Anhang E) eine Schritt-für-Schritt-Anleitung zur Durchführung der notwendigen Arbeitsschritte ergänzt. ORS Tools liefert in Kombination mit Google Maps sehr anschauliche und nachvollziehbare Ergebnisse, bringt jedoch einen höheren manuellen Aufwand mit sich, da die Berechnungen nicht automatisiert ablaufen. Der Python-basierte Ansatz bietet hingegen den Vorteil, dass er sich in eine automatisierte Pipeline integrieren lässt und ohne manuelle Zwischenschritte ausgeführt werden kann. Durch die Verwendung von Variablen und einer klaren Struktur kann diese Lösung zudem relativ einfach angepasst und auch für zukünftige Transportunternehmen oder weitere Anwendungsfälle eingesetzt werden.

7 Beantwortung der Forschungsfragen

Die in Kapitel 1.2 formulierten Forschungsfragen können nach der durchgeführten Analyse wie folgt beantwortet werden:

Die GTFS-Daten der Fernverkehrshaltestellen können technisch auf unterschiedliche Weisen verarbeitet und in das System *opentransportdata.swiss* integriert werden. Einerseits stehen dafür verschiedene Werkzeuge in QGIS zur Verfügung, andererseits lassen sich diese Arbeitsschritte auch mithilfe von Python-Skripten umsetzen. Während QGIS vor allem für visuelle Analysen und einzelne Verarbeitungsschritte geeignet ist, zeigt sich bei komplexeren ETL-Prozessen, dass diese dort nur mit grossem Aufwand und kaum automatisiert durchgeführt werden können. In Python hingegen lassen sich diese Schritte klar strukturieren, automatisieren und wiederholbar ausführen, was die Integration der GTFS-Daten deutlich effizienter macht.

Die Berechnung der Fusswegdistanzen und Umsteigezeiten zwischen Fernverkehrshaltestellen und bestehenden öV-Haltestellen erfolgt bei allen durchgeführten Methoden auf Basis des tatsächlichen Gehwegs. Es wird also nicht einfach die Luftliniendistanz verwendet, sondern der reale Weg, den eine Person zu Fuss zurücklegen muss. Sowohl ORS Tools als auch OSMnx greifen dabei auf Daten aus OSM zurück, welche weltweit verfügbare Informationen zu Strassen, Gehwegen und Fusswegen enthalten. QNEAT3 hingegen nutzt mit dem swissTLM3D-Datensatz von swisstopo ein sehr detailliertes, speziell für die Schweiz erstelltes Wegenetz. Die auf diese Weise berechneten Gehstrecken werden anschliessend in Gehminuten umgerechnet und dienen als Grundlage für die Bestimmung der Umsteigezeiten.

Als zentrales Kriterium für eine sinnvolle Umsteigebeziehung wurde in dieser Arbeit die Gehzeit definiert. Zusätzlich wurde ein fixer Zeitaufschlag von zwei Minuten berücksichtigt, um den Zeitbedarf für das Be- und Entladen von Gepäck bei Fernverkehrsbussen realistisch abzubilden. Die Kombination aus Gehzeit und dem festgelegten Zuschlag bildet somit die Entscheidungsgrundlage für die Bewertung einer Umsteigebeziehung.

Um die nächstgelegene öV-Haltestelle zu einer Fernverkehrshaltestelle geografisch zu ermitteln, werden zunächst die Koordinaten beider Haltestellentypen herangezogen. Jede Fernverkehrshaltestelle wird dabei als Ausgangspunkt betrachtet, während die bestehenden öV-Haltestellen als mögliche Zielpunkte dienen. In einem ersten Schritt wird die Luftliniendistanz zu allen umliegenden öV-Haltestellen berechnet, um eine räumliche Vorauswahl zu treffen. Anschliessend wird für diese ausgewählten Haltestellen der tatsächliche Fussweg auf dem jeweiligen Wegenetz berechnet. Die öV-Haltestelle mit der kürzesten Gehzeit wird schliesslich als nächstgelegene Haltestelle definiert. Unabhängig von der verwendeten Methode folgt dieser Ablauf immer demselben Prinzip, auch wenn sich die Datengrundlagen und Berechnungsansätze im Detail unterscheiden.

8 Ergebnis und Ausblick

Die Ergebnisse der Arbeit zeigen, dass die Fernverkehrshaltestellen auf unterschiedliche Weisen anhand einer Umsteigebeziehung mit den öV-Haltestellen verknüpft und in das System *opentransportdata.swiss* integriert werden können. Einerseits stehen dafür verschiedene Werkzeuge in QGIS zur Verfügung, andererseits lassen sich diese Arbeitsschritte auch mithilfe von Python-Skripten umsetzen.

Anhand des Methodenvergleichs zeigt sich, dass zur Berechnung der Umsteigebeziehung entweder der automatisierte Python-Code oder die ORS-Tools-Methode in QGIS eingesetzt werden kann. Ein klarer Vorteil des Python-Ansatzes liegt in der Möglichkeit, diesen in eine automatisierte Pipeline zu integrieren. Dadurch kann der gesamte Ablauf ohne manuelle Eingriffe durchgeführt werden, was insbesondere für zukünftige Anwendungen oder weitere Transportunternehmen von Vorteil ist. Die ORS-Tools-Methode liefert in Kombination mit Google Maps zwar anschauliche und gut nachvollziehbare Ergebnisse, bringt jedoch einen höheren manuellen Aufwand mit sich und eignet sich daher weniger für eine automatisierte Umsetzung.

Ein möglicher Ausblick besteht darin, die Python-Lösung in eine bestehende Pipeline zu integrieren. Dies kann durch das automatische Hochladen der berechneten Resultate per Secure File Transfer Protocol (SFTP) erfolgen, sodass diese anschliessend von *opentransportdata.swiss* geharvestet werden können. Auf diese Weise kann der neu erzeugte Datensatz online zur Verfügung gestellt und künftig regelmässig aktualisiert werden.

Eine weitere Möglichkeit, um die Validität der berechneten Umsteigezeit zu überprüfen, gleich wie der Vergleich mit Google Maps, ist der Abgleich mit der sogenannten Grundwahrheit. Unter Grundwahrheit werden überprüfte und möglichst genaue Daten verstanden, die als Referenz für den Vergleich mit berechneten oder modellierten Ergebnissen dienen. Diese Daten basieren auf realen Beobachtungen und gelten daher als besonders verlässlich. Sie werden genutzt, um zu beurteilen, wie gut ein Modell oder eine Berechnung die tatsächliche Situation abbildet (Krantz & Jonker, o. J.). Im Zusammenhang mit den Umsteigebeziehungen bedeutet dies, dass ausgewählte Gehwege, insbesondere jene, bei denen im Methodenvergleich Unterschiede festgestellt wurden, tatsächlich vor Ort zu Fuss zurückgelegt werden. Die dabei gemessenen Gehzeiten können anschliessend mit den berechneten Zeiten verglichen werden. Auf diese Weise lässt sich beurteilen, wie gut die verwendeten Methoden die reale Situation abbilden und ob die berechneten Umsteigezeiten plausibel sind.

Damit die Umsteigebeziehungen noch genauer berechnet werden, könnte zusätzlich die Höheninformation über die Z-Koordinate mit einbezogen werden. Gerade beim Vergleich mit Google Maps ist dies sinnvoll, da dort Höhenunterschiede bereits in die Berechnung der Gehzeiten berücksichtig werden. Eine mögliche Umsetzung bietet die Google Maps Elevation API (Application Programming Interface). Mit dieser können Höheninformationen entlang einer Route zwischen zwei Punkten abgefragt werden, indem die Breiten- und Längengradkoordinaten als Folge von Punkten entlang eines Pfades übergeben werden (Google Maps Platform, 2025). Diese API kann direkt in den Python-Code integriert werden.

Literaturverzeichnis

Boeing, G. (2025). Wiley Online Library. Geographical Analysis. https://doi.org/10.1111/GEAN.70009;PAGE:STRING:ARTICLE/CHAPTER

Daten für ein effizientes Mobilitätssystem - BAV. (2025, Juni 10). https://www.bav.admin.ch/de/daten-fuer-effizientes-mobilitaetssystem

dges.carleton.ca. (2021, September 30). https://dges.carleton.ca/CUOSGwiki/index.php/Network_Analysis%3A_Finding_shortest_path_in_QGIS

Entdecken Sie BlaBlaCar Bus. (o. J.). Abgerufen 11. Dezember 2025, von https://www.blablacar.de/bus/companies/blablacar-bus

Gandhi, U. (2025, Oktober 31). QGIS Tutorials and Tips. https://www.qgistutorials.com/de/docs/3/service_area_analysis.html

GitHub-Dokumentation. (o. J.). Abgerufen 11. Dezember 2025, von https://docs.github.com/de/get-started/start-your-journey/about-github-and-git#informationen-zu-github

Google Maps Platform. (2025, Dezember 16). https://developers.google.com/maps/documentation/elevation/overview?utm_source=chatgpt.com&hl=de

Heller, M. (2024, Dezember 4). Computerwoche. https://www.computerwoche.de/article/3611333/was-ist-visual-studio-code.html

HRDF-Realisierungsvorgaben-öV-Schweiz. (2024).

Krantz, T., & Jonker, A. (o. J.). IBM. Abgerufen 18. Dezember 2025, von https://www.ibm.com/de-de/think/topics/ground-truth

maptransfer. (o. J.). Was ist QGIS? | Maptransfer. Abgerufen 7. November 2025, von https://www.maptransfer.de/was-ist-qgis

Oev-info. (2020). 3. https://www.oev-info.ch/sites/default/files/2023-03/datenmanagement_umsteigezeit_transfertzeit_ski_v.0.3_de.pdf

opendata.swiss. (2024, Dezember 18). https://opendata.swiss/de/dataset/swissboundaries3d-kantonsgrenzen

opentransportdata.swiss. (o. J.). Abgerufen 17. Dezember 2025, von https://opentransportdata.swiss/de/cookbook/timetable-cookbook/hafas-rohdaten-format-hrdf/

Oracle. (o. J.). Abgerufen 11. Dezember 2025, von https://www.oracle.com/de/developer/what-is-python-for-developers/

Plugins QGIS. (o. J.). Abgerufen 16. Dezember 2025, von https://plugins.qgis.org/plugins/ORStools/

QGIS. (o. J.). Abgerufen 16. Dezember 2025, von https://qgis.org/download/

Raffler, C. (2018a). root 676, QGIS Network Analysis Toolbox 3. https://root676.github.io/

Raffler, C. (2018b). root 676, Shortest Path Alogrithm. https://root676.github.io/ShortestPathAlgs.html

Services | openrouteservice. (o. J.). Abgerufen 14. Dezember 2025, von https://openrouteservice.org/services/

swisstopo.admin. (2024, März 8). https://www.swisstopo.admin.ch/de/landschaftsmodell-swisstlm3d

Über Flix: Fernbus- und Zugbetreiber. (o. J.). Abgerufen 11. Dezember 2025, von https://www.flixbus.ch/unternehmen/ueber-flixbus

Über SKI+ | öv-info.ch. (o. J.). Abgerufen 11. Dezember 2025, von https://www.oev-info.ch/de/organisation/ueber-ski

VÖV UTP. (2024).

Anhang

A - QNEAT3 `METABHF` Table

Start	Ziel	QNEAT3 Gehminuten	GoogleMaps Gehminuten
1700000	8570835	1	1
1700001	8510623	0	1
1700002	8577579	0	1
1700003	8575224	1	1
1700004	8573205	2	1
1700005	8502056	0	1
1700006	8577394	22	30
1700007	8577245	19	22
1700008	8589852	2	2
1700009	8509884	0	1
1700010	8596126	1	1
1700011	8581044	1	1
1700012	8500189	3	3
1700013	8507226	0	1
1700014	8589611	4	3
1700015	8592088	1	1
1700016	8590075	4	2
1700017	8774890	0	1
1700018	8594298	1	1
1700019	8519133	3	3
1700020	8501292	1	1
1700021	8502058	6	6
1700022	8581820	0	1
1700023	8592679	1	1
1700024	8592883	1	1
1700025	8582639	2	2
1700026	8583270	3	2
1700027	8500146	2	3
1700028	8589139	1	1
1700029	8502058	1	2

B - ORS Tools `METABHF` Tabelle

Start	Ziel	ORS Tools Gehminuten	GoogleMaps Gehminuten
1700000	8570835	1	1
1700001	8510623	0	1
1700002	8577579	0	1
1700003	8575224	1	1
1700004	8573205	1	1
1700005	8502056	0	1
1700006	8577381	26	22
1700007	8577245	30	22
1700008	8589852	2	2
1700009	8509884	0	1
1700010	8596126	0	1
1700011	8581044	1	1
1700012	8500189	3	3
1700013	8507226	0	1
1700014	8589611	3	3
1700015	8592088	1	1
1700016	8590075	2	2
1700017	8774890	0	1
1700018	8594298	1	1
1700019	8589164	1	1
1700020	8501292	0	1
1700021	8502058	6	6
1700022	8581820	2	1
1700023	8592679	1	1
1700024	8592883	1	1
1700025	8582639	2	2
1700026	8501506	2	1
1700027	8500146	2	3
1700028	8589139	1	1
1700029	8502058	1	2

C - OSMnx `METABHF` Tabelle

Start	Ziel	OSMnx Gehminuten	Google Maps Gehminuten
1700000	8570835	1	1
1700001	8510623	0	1
1700002	8577579	0	1
1700003	8575224	1	1
1700004	8573205	1	1
1700005	8502056	0	1
1700006	8530414	27	38
1700007	8577245	19	22
1700008	8589852	1	2
1700009	8509884	0	1
1700010	8591368	1	2
1700011	8575112	1	2
1700012	8500189	3	3
1700013	8507226	0	1
1700014	8589602	4	8
1700015	8592088	1	1
1700016	8590075	2	2
1700017	8774890	0	1
1700018	8594298	1	1
1700019	8589164	1	1
1700020	8501292	0	1
1700021	8589847	6	8
1700022	8581820	2	1
1700023	8592679	0	1
1700024	8592883	1	1
1700025	8582639	1	2
1700026	8501506	2	1
1700027	8500146	2	3
1700028	8589139	3	1
1700029	8502058	2	2

D - R-Code der Analyse

```
# Pakete (einmalig installieren, dann auskommentieren)
# install.packages(c("readxl", "dplyr", "ggplot2", "tidyr",
"scales"))


library(readxl)
library(dplyr)
library(tidyr)
library(ggplot2)
library(scales)


path <- "C:/Users/imeri/OneDrive - Hochschule Luzern/Masterar-
beit/tests/report_export.xlsx"


sheet_id <- 4 # 2, 3 oder 4


method_name <- "OSMnx"   # wenn sheet-id 2 dann "QNEAT, wenn 3
dann "ORS", wenn 4 dann "OSMnx"


col_model  <- paste(method_name, "Gehminuten")
col_google <- "GoogleMaps Gehminuten"


df_raw <- read_excel(path, sheet = sheet_id)


df <- df_raw %>%
  rename(
    methode_min  = all_of(col_model),
    google_min = all_of(col_google)
  )


# 1) Daten einlesen + Kennzahlen berechnen
df <- df %>%
  mutate(
    `Start -> Ziel` = paste(Start, "→", Ziel),
    delta     = methode_min - google_min,
    abs_delta = abs(delta),
    pct_delta = if_else(google_min == 0, NA_real_, delta /
google_min),
    faktor    = if_else(google_min == 0, NA_real_, methode_min /
google_min),
```

```
    ok_2min   = abs_delta <= 2,
    delta_klasse = case_when(
      abs_delta <= 1 ~ "≤ 1 Minute",
      abs_delta <= 3 ~ "2–3 Minuten",
      TRUE           ~ ">= 4 Minuten"
    )
  )

# Sortierreihenfolge nach col_model (für Plot 1)
order_levels <- df %>%
  arrange(methode_min) %>%
  pull(`Start -> Ziel`)

# 2) Long-Format für Plot 1 (col_model vs Google)
df_long <- df %>%
  select(`Start -> Ziel`, methode_min, google_min) %>%
  pivot_longer(
    cols = c(methode_min, google_min),   # ← Reihenfolge hier!
    names_to = "quelle",
    values_to = "minuten"
  ) %>%
  mutate(
    `Start -> Ziel` = factor(`Start -> Ziel`, levels = order_le-
vels),
    quelle = recode(
      quelle,
      methode_min = method_name,
      google_min  = "Google Maps"
    ),
    quelle = factor(quelle, levels = c(method_name, "Google
Maps"))
  )

p_vergleich <- ggplot(df_long, aes(minuten, `Start -> Ziel`,
fill = quelle)) +
  geom_col(position = position_dodge2(width = 0.9, reverse =
TRUE)) +
  labs(
    title = paste("Vergleich Gehminuten:", method_name, "vs.
Google Maps"),
    x = "Gehminuten",
```

```
    y = "Start → Ziel",
    fill = "Quelle"
  ) +
  theme(axis.text.x = element_text(angle = 45, hjust = 1))

p_vergleich

# 3) Delta-Plot (sortiert nach delta)
df_delta <- df %>%
  arrange(delta) %>%
  mutate(`Start -> Ziel` = factor(`Start -> Ziel`, levels =
`Start -> Ziel`))

p_delta <- ggplot(df_delta, aes(delta, `Start -> Ziel`, fill =
delta > 0)) +
  geom_col(show.legend = FALSE) +
  geom_hline(yintercept = 0) +
  labs(
    title = paste("Delta (", method_name, "- Google Maps)", sep
= ""),
    x = "Start → Ziel",
    y = "Delta (Minuten)"
  ) +
  theme(axis.text.x = element_text(angle = 45, hjust = 1))

p_delta

# 4) Benötigte Measures / Auswertungen
delta_counts <- df %>%
  count(delta_klasse)

delta_shares <- df %>%
  count(delta_klasse) %>%
  mutate(anteil_pct = n / sum(n) * 100)

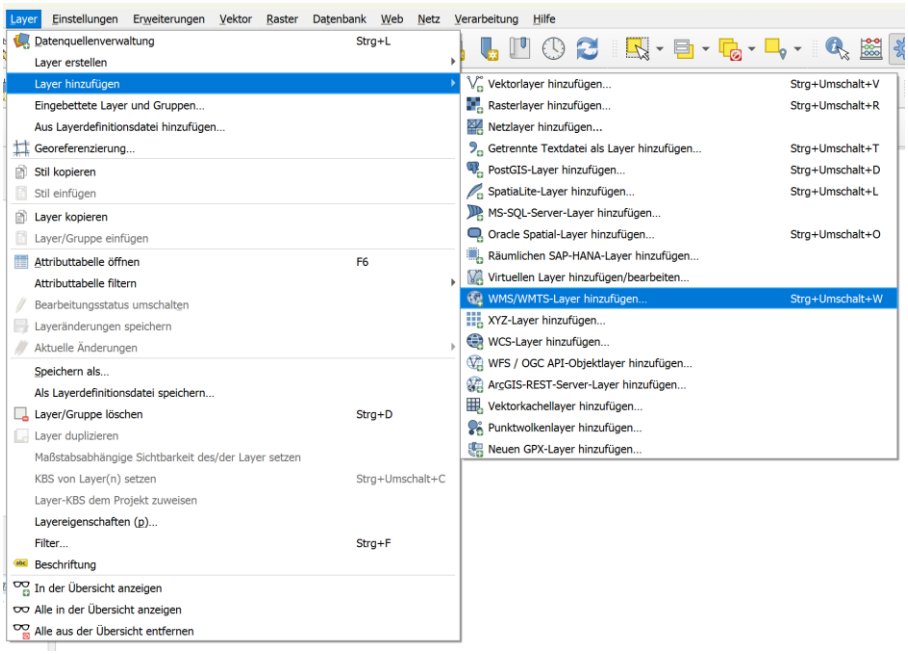
delta_summary <- df %>%
  summarise(
    mean_abs_delta_min   = mean(abs_delta, na.rm = TRUE),
    median_abs_delta_min = median(abs_delta, na.rm = TRUE),
    max_abs_delta_min    = max(abs_delta, na.rm = TRUE)
```

```
  )


delta_counts
delta_shares
delta_summary
```

E - Anleitung für ORS Tools zur Berechnung der Umsteigezeit in QGIS

1) Hintergrundkarte laden (swisstopo)
   - **Layer → Layer hinzufügen → WMS/WMTS-Layer hinzufügen…**

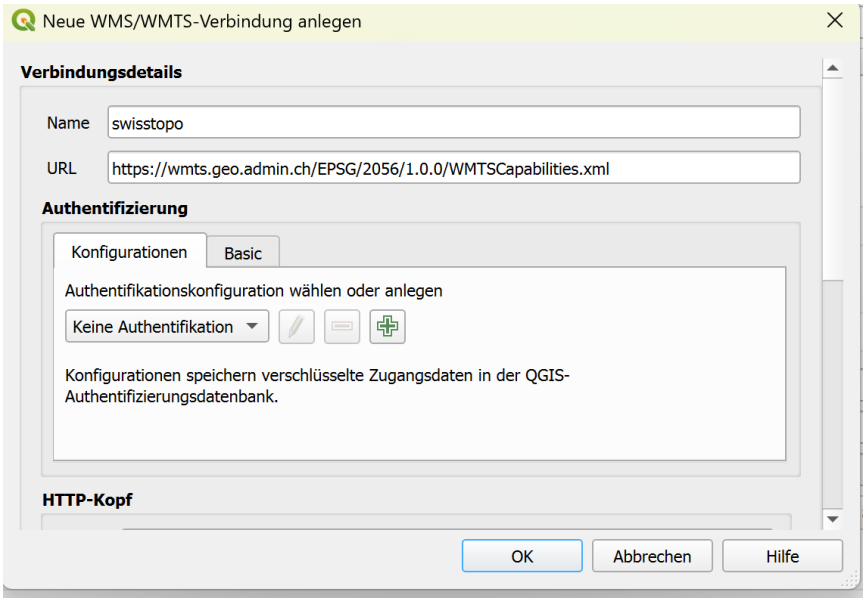


(a)

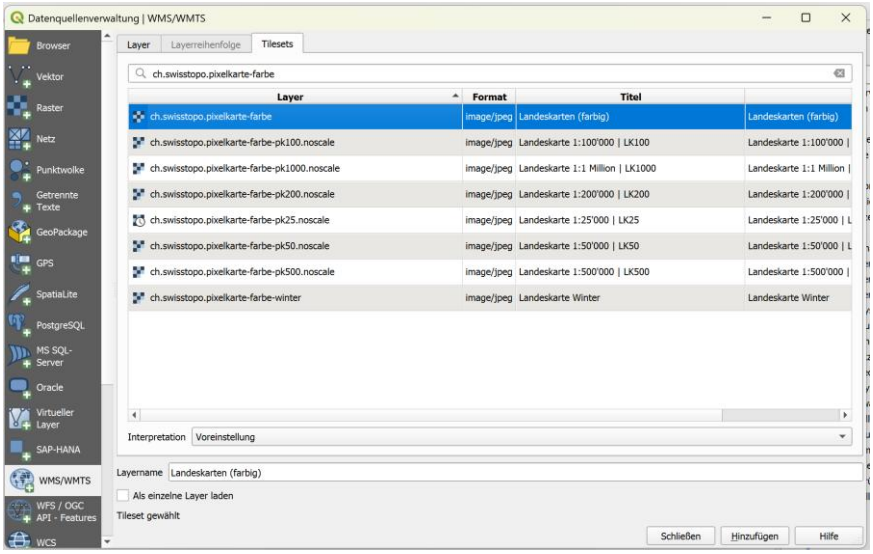
2) WMTS-Verbindung anlegen
   - **Neu**

     Name: swisstopo

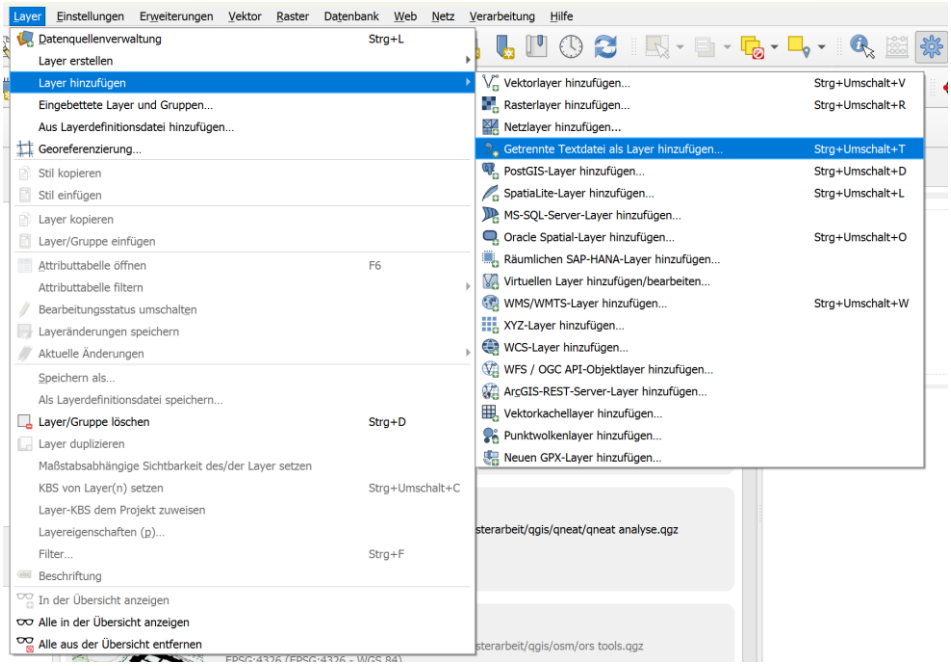
     URL: https://wmts.geo.admin.ch/EPSG/2056/1.0.0/WMTSCapabilities.xml



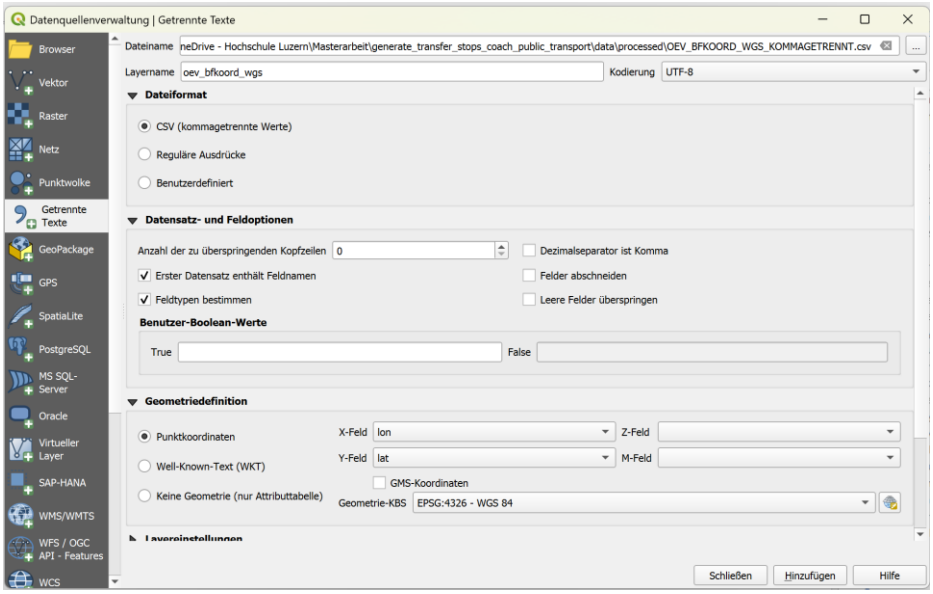
3) Karte hinzufügen
   - **Verbinden → Tileset "ch.swisstopo.pixelkarte-farbe" wählen → hinzufügen**



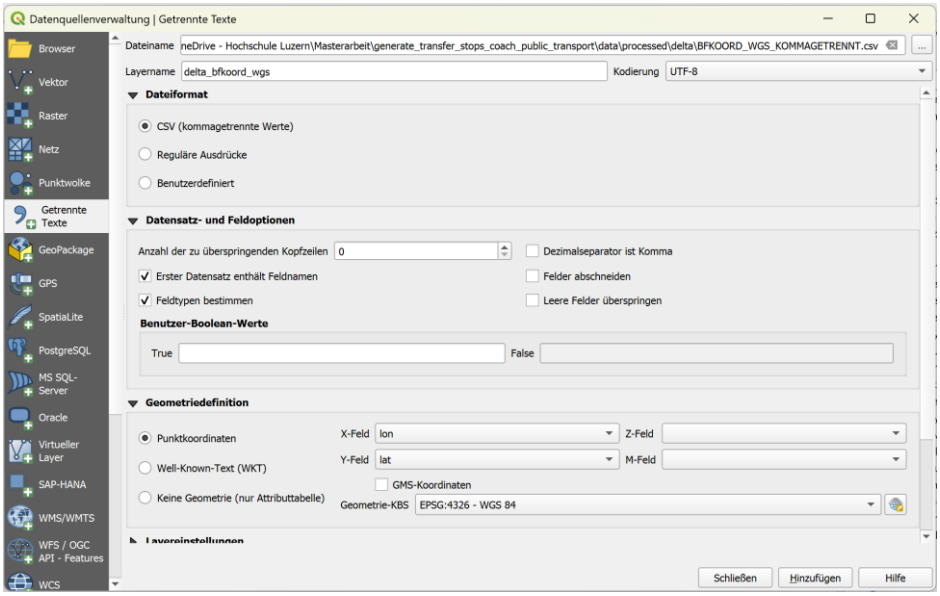
4) CSV-Layer laden (Haltestellen)
   - Register Layer → **Layer hinzufügen → Getrennte Textdatei als Layer hinzufügen …**



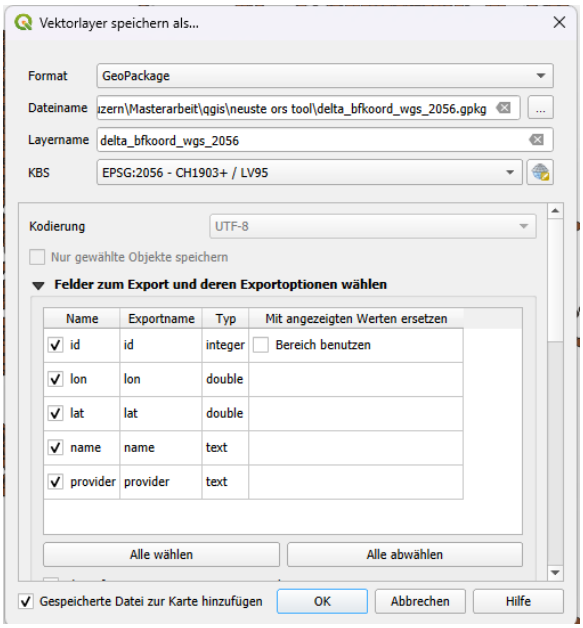
5) ÖV-Haltestellen laden
   - **Datei:** OEV_BFKOORD_WGSKOMMAGETRENNT
   - **Layername:** oev_bfkoord_wgs



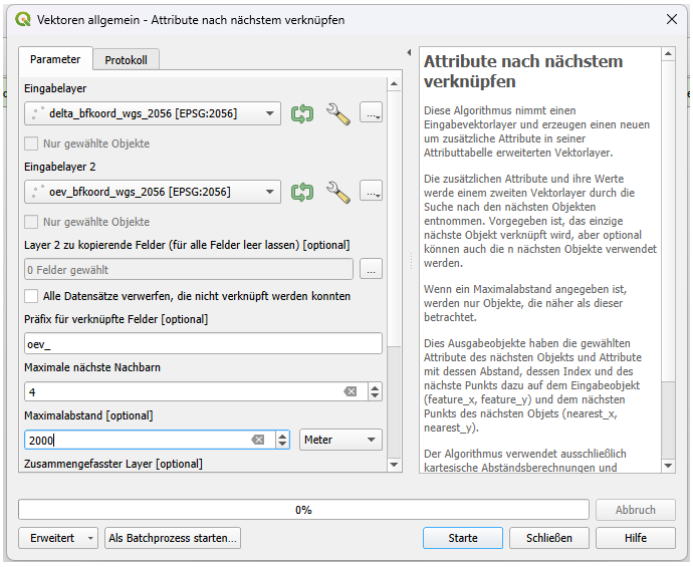
6) Fernverkehr-Haltestellen laden
   - **Datei:** BFKOORD_WGS_KOMMAGETRENNT.csv
   - **Layername:** delta_bfkoord_wgs



7) Beide Layers nach ESPG:2056 speichern (Reprojektion)
   - Rechtsklick auf Layer → **Export → Objekt speichern als…**
     i) **Format:** GeoPackage
     ii) **KBS:** EPSG:2056
     iii) Dateiname → drei punkte → als gpkg speichern → **delta_bfkoord_wgs.gpkg**
     iv) **Layername:** delta_bfkoord_wgs_2056
     v) **KBS:** EPSG:2056

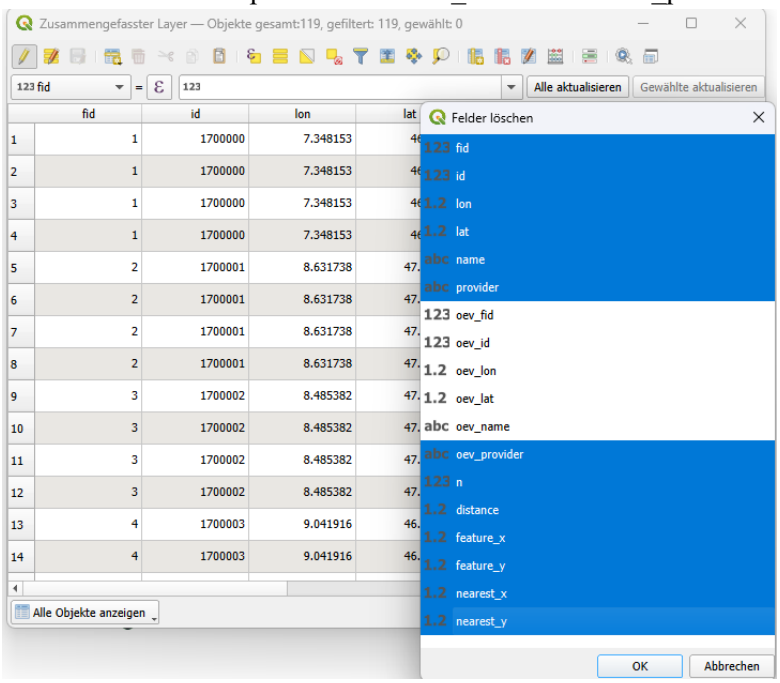


8) Das gleiche für oev_bfkoord_wgs
   Dateiname: oev_bfkoord_wgs.gpkg
   Layername: oev_bfkoord_wgs_2056
   KBS: EPSG:2056
9) Pro Fernverkehrshaltestelle nur die nächsten öV-Haltestellen holen
   - Verarbeitungswerkzeuge → **Attribute nach nächstem verknüpfen**
     Input Layer: delta_bfkoord_wgs_2056
     Input Layer 2: oev_bfkoord_wgs_2056
     Präfix: oev_
     Max. Nachbarn: 4
     Max. Abstand: 2000 m



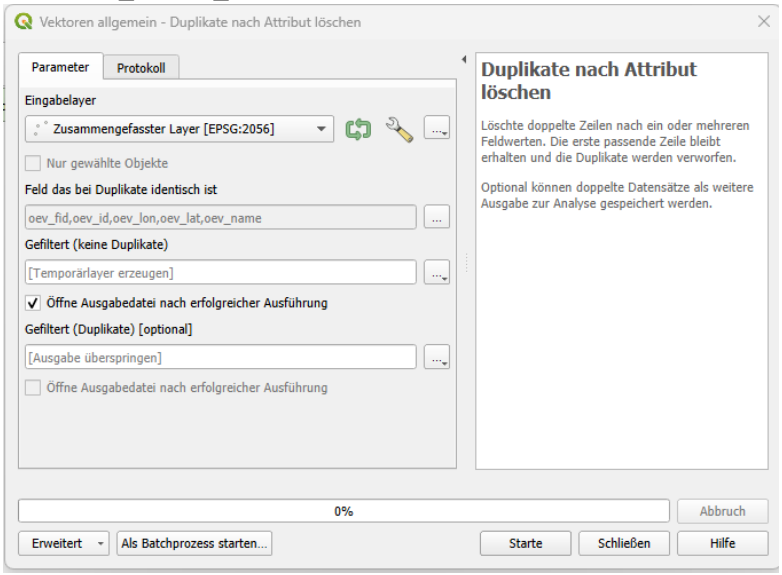
10) Spalten aufräumen (nur oev_behalten)

- Rechtsklick auf **Zusammengefasster Layer → Bearbeitungsmodus (Bleistift) → Attributstabelle →** alle Spalten ausser oev_* löschen + oev_provider



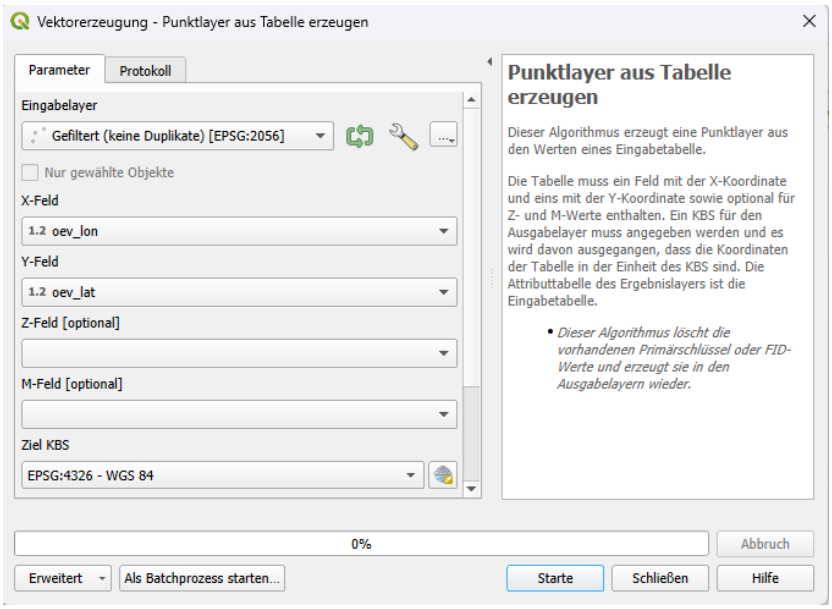
11) Doppelte öV-Haltestellen entfernen

- Verarbeitungswerkzeug → **Duplikat nach Attribut löschen**
  Eingabelayer: Zusammengefasster Layer
  Feld: oev_id, oev_fid



12) Neue Punktlayer aus oev-Koordinaten erstellen

- Verarbeitungswerkzeug → **Punktlayer aus Tabelle erzeugen:**
  Eingabelayer: Gefiltert (keine Duplikate)
  X-Feld: oev_lon
  Y-Feld: oev_lat
  Ziel KBS: EPSG:4326

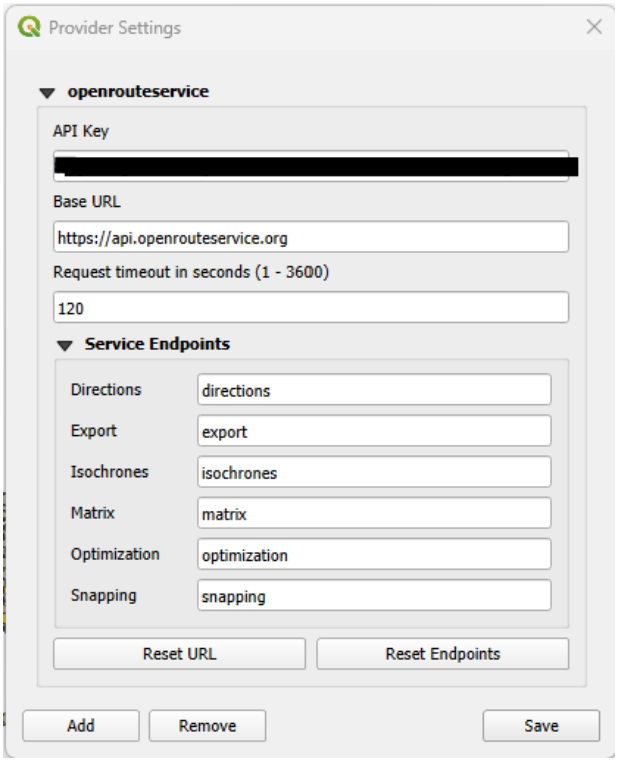


13) ORS Tools installieren

- Erweiterung → **Erweiterungen verwalten und installieren...** → nach **ORS Tools** suchen → **installieren.**

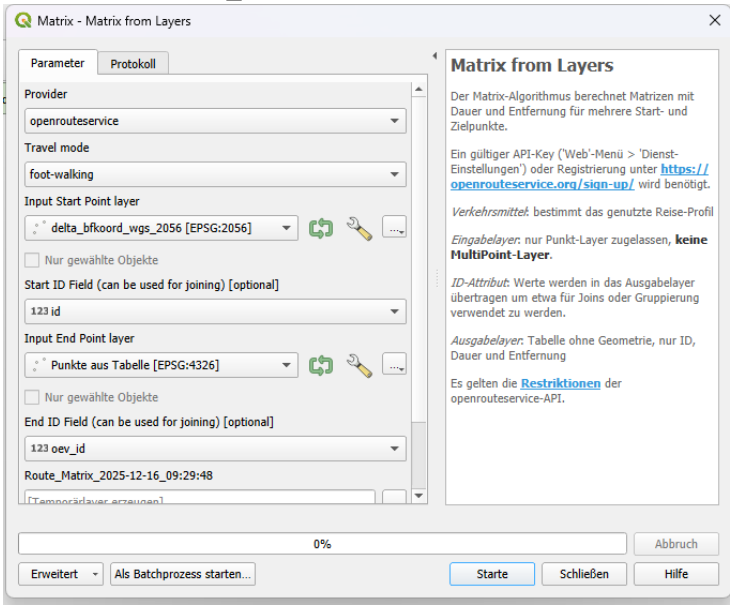
14) API-Key setzen

- Auf der **openrouteservice-Webseite** einloggen → **API Key kopieren**
- **In QGIS:** Web → **ORS Tools** → **Provider Settings** → Key einfügen



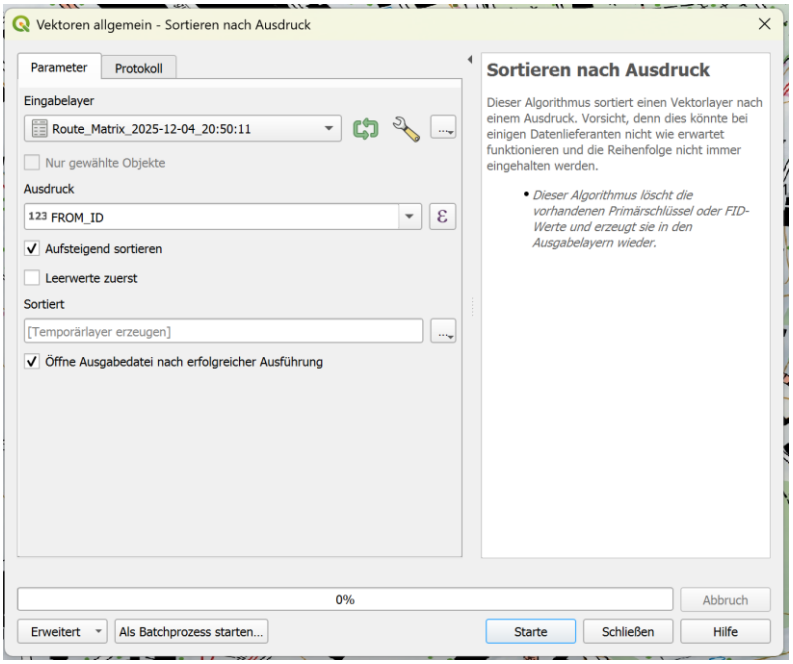
15) Matrix berechnen

- Verarbeitungswerkzeug → **ORS Tools → Matrix from Layers:**
  Provider: Openrouteservice
  Travel mode: foot-walking
  Input Start Point layer: delta_bfkoord_wgs_2056
  Start ID Field: id
  Input End Point layer: Punkte aus Tabelle
  End ID Field: oev_id



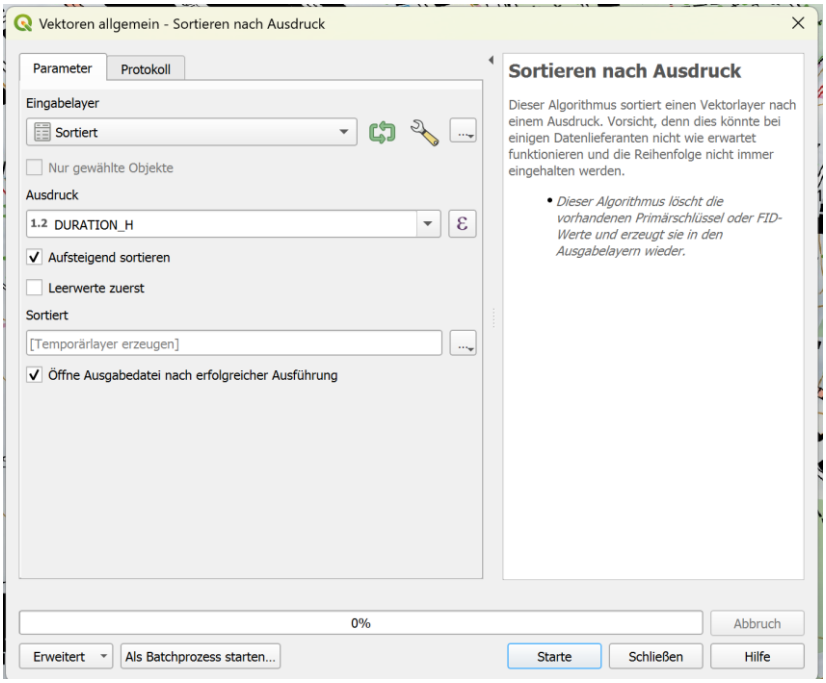
16) Nach FROM_ID sortieren

- Verarbeitungswerkzeug → **Sortieren nach Ausdruck**
  Eingabelayer : Route_Matrix_…
  Ausdruck: FROM_ID



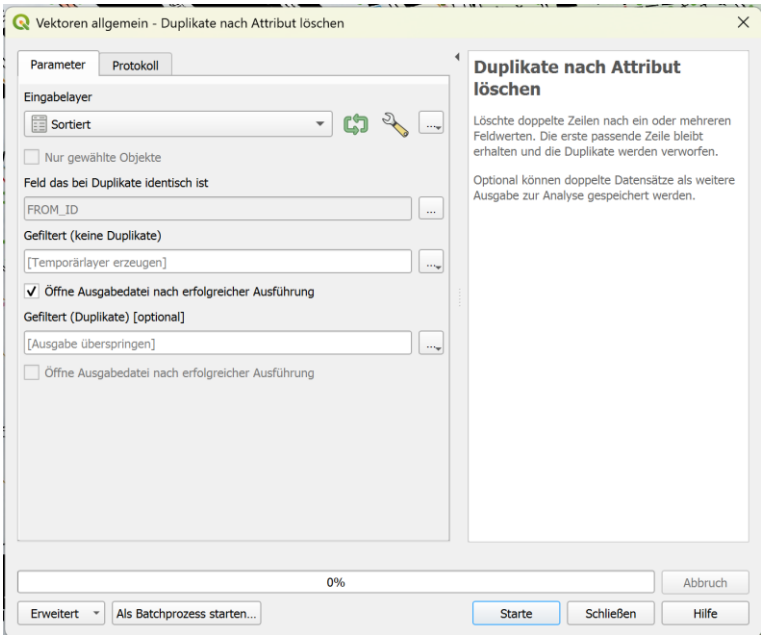
17) Danach nach Dauer sortieren (kleinste zuerst)

- Verarbeitungswerkzeug → **Sortieren nach Ausdruck**
  Eingabelayer : Sortiert
  Ausdruck: DURATION_H



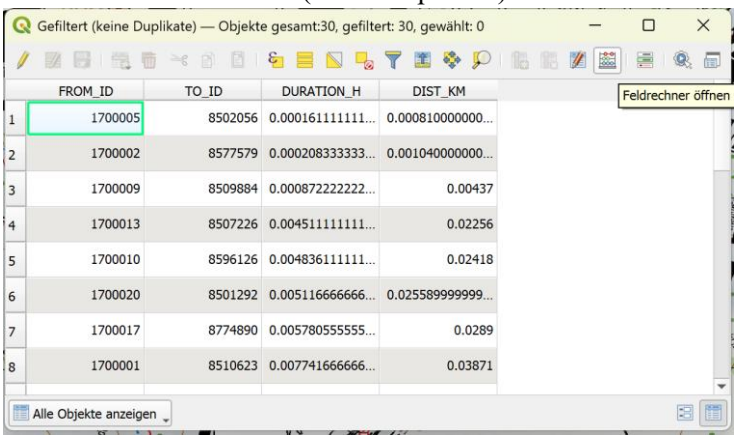
18) Pro FROM__ID nur den ersten Eintrag behalten

- Verarbeitungswerkzeug → **Duplikate nach Attribut löschen**
  Eingablayer: Sortiert (das neuste)
  Feld das bei Dupliakte identisch ist: FROM_ID



19) Minutenfeld berechnen (inkl. +2 Minuten)

- Rechtsklick auf Gefiltert (keine Duplikate) → **Attributtabelle → Feldrechner**



	FROM_ID	TO_ID	DURATION_H	DIST_KM
1	1700005	8502056	0.000161111111...	0.000810000000...
2	1700002	8577579	0.000208333333...	0.001040000000...
3	1700009	8509884	0.000872222222...	0.00437
4	1700013	8507226	0.004511111111...	0.02256
5	1700010	8596126	0.004836111111...	0.02418
6	1700020	8501292	0.005116666666...	0.025589999999...
7	1700017	8774890	0.005780555555...	0.0289
8	1700001	8510623	0.007741666666...	0.03871

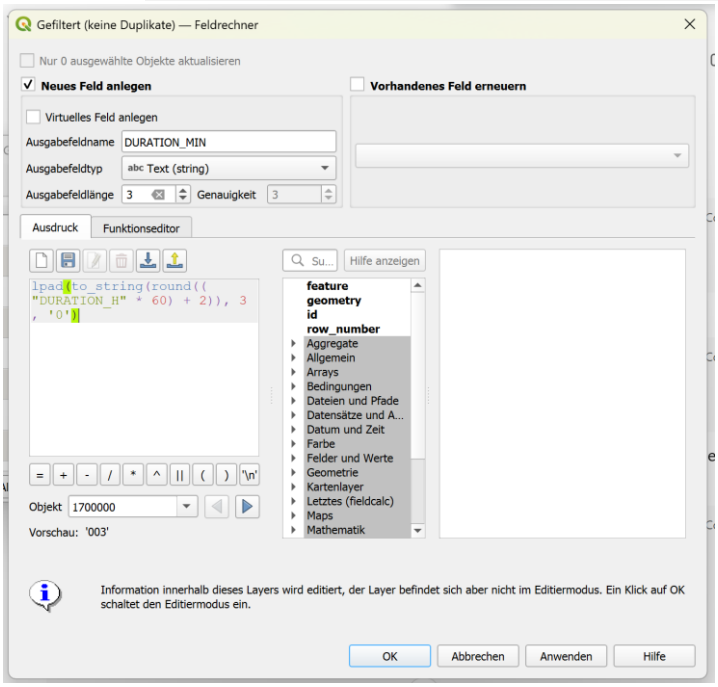
Neues Feld: *DURATION_MIN*

Ausgabefeldname: DURATION_MIN

Ausgabefeldtyp: Text (string)

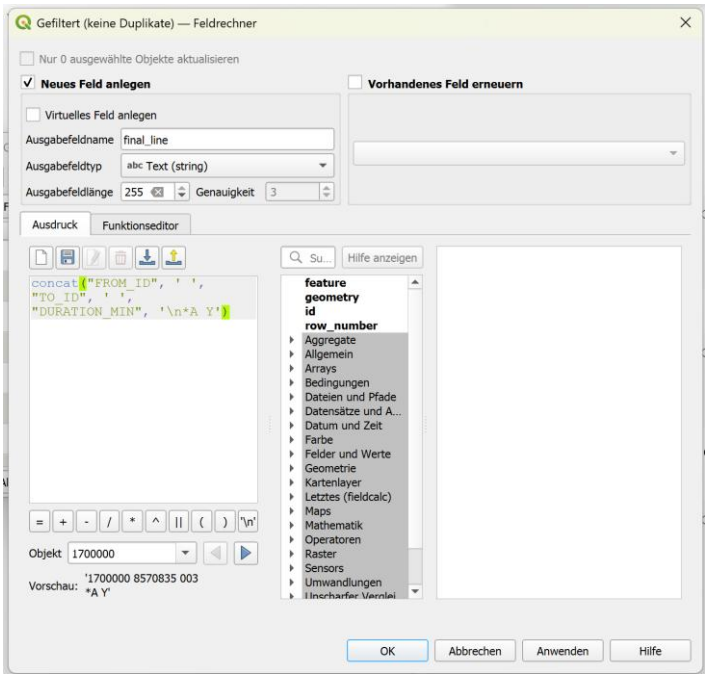
Ausgabefeldlänge: 3

Ausdruck: `lpad(to_string(round(("DURATION_H" * 60) + 2)), 3, '0')`



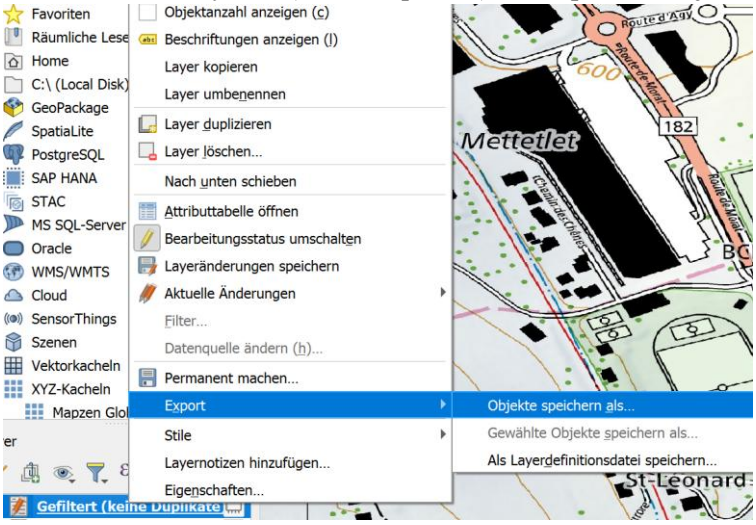
20) Neues Feld für Exportzeile: *final_line*

- **Ausgabefeldname:** final_line
  Ausgabefeldtyp: Text (string)
  Ausgabefeldlänge: 100
  Ausdruck: `concat("FROM_ID",' ', "TO_ID", ' ', "DURATION_MIN", '\n*A Y')`



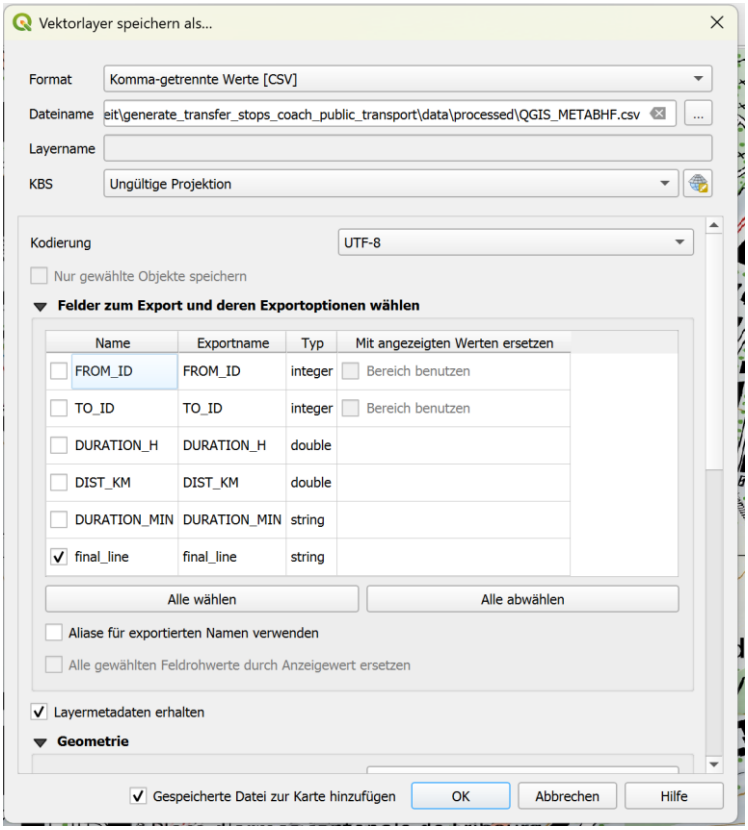
21) Export als CSV

- Rechtklick auf *Gefiltert (keine Duplikate)* → Export → Objekte speichern als …



22) Export-Einstellungen

- **Format:** Komma-getrennte Werte [CSV]
  Dateiname: in den Ordner der sich im repo befindet: Data/processed/ und name QGIS_METaF.csv
  Felder zum Export und deren Exportoptionen wählen: nur final_line auswählen



23) Zum Schluss: Python-Processing im Repo

- **Script ausführen:** process_delta_metabhf.csv (wie im Projekt vorgesehen)

Eidesstattliche Erklärung

Hiermit versichere ich an Eides statt und durch meine Unterschrift, dass die vorliegende Arbeit von mir selbstständig ohne fremde Hilfe angefertigt worden ist. Inhalte und Passagen, die aus fremden Quellen stammen und direkt oder indirekt übernommen worden sind, wurden als solche kenntlich gemacht. Ferner versichere ich, dass ich keine andere ausser der im Literaturverzeichnis angegebenen Literatur verwendet habe. Diese Versicherung bezieht sich sowohl auf Textinhalte als auch auf alle enthaltenden Abbildungen, Skizzen und Tabellen. Die Arbeit wurde bisher keiner Prüfungsbehörde vorgelegt und auch noch nicht veröffentlicht.

Ort, Datum Bern, 19.12.2025

Unterschrift